Cheap Multi-Robot Autonomous Fleets for Disaster Recovery: Phone-as-Brain Rovers with Depth Sensing, Obstacle Avoidance, and Fleet Command Interfaces

Yojan Gautam

October 27, 2026

Contents

Introduction **5**

1 Background and Related Work **10**

1.1 Robots in disaster response . . . 10

1.1.1 World Trade Center, 2001 . . . 10

1.1.2 Fukushima Daiichi, 2011 . . . 10

1.1.3 DARPA Subterranean Challenge, 2021 . . . 11

1.1.4 Champlain Towers South, Surfside, 2021 . . . 11

1.1.5 Recurring lessons . . . 11

1.2 Levels of autonomy . . . 11

1.3 Supervising multiple robots . . . 12

1.3.1 Neglect tolerance and fan-out . . . 12

1.3.2 Attention, workload, and situation awareness . . . 13

1.4 Operator interfaces for robot fleets . . . 13

1.5 Communication constraints . . . 13

1.6 Low-cost onboard navigation . . . 14

1.6.1 Local planning . . . 14

1.6.2 Phones as robot computers . . . 14

1.7 Multi-robot coordination . . . 14

1.8 Gaps addressed by this thesis . . . 14

2 The NEXT Competition and the Experimental Environment **16**

2.1 Why a competition . . . 16

2.2 The NEXT Competition 2026 . . . 16

2.2.1 Match format . . . 17

2.2.2 Field and game objects . . . 17

2.2.3 Scoring and penalties . . . 17

2.2.4 What the competition asks of a robot . . . 17

2.3 The experimental environment . . . 17

2.3.1 The router . . . 18

2.3.2 The phone as the brain . . . 19

2.3.3 The Pi motor bridge . . . 19

2.3.4 The operator station . . . 20

2.3.5 Outdoor GPS navigation and heading . . . 20

2.3.6 Frontier exploration . . . 21
2.3.7 Zorvane: the NEXT field twin . . . 21
2.3.8 Logging and measurement . . . 22
2.4 From the field to the real situation . . . 22
2.4.1 The lessons from Chapter 1, on the field . . . 22
2.4.2 Where the analogy breaks . . . 24
2.5 Study conditions on the NEXT field . . . 24
2.6 Summary . . . 24

3 Rover Physical Design **26**
3.1 Design requirements from the rules . . . 26
3.2 Two chassis . . . 27
3.2.1 The Terra development chassis . . . 27
3.2.2 CrabTrapBot . . . 27
3.3 Ball handling . . . 28
3.4 The phone mount . . . 28
3.5 Drive electronics . . . 29
3.5.1 The Pi motor bridge . . . 29
3.5.2 Safety in the hardware path . . . 30
3.5.3 Skid-steer turning . . . 30
3.5.4 Rover profiles . . . 30
3.6 Power . . . 31
3.7 Planned: a USB microcontroller in place of the Pi . . . 31
3.8 Open measurements . . . 32

4 Simulation **33**
4.1 Role in the thesis . . . 33
4.2 Why a custom simulator . . . 34
4.3 Architecture . . . 34
4.3.1 Vehicle bodies . . . 34
4.3.2 Physics and drive . . . 35
4.3.3 Sensors . . . 35
4.3.4 The bridge . . . 35
4.3.5 Game logic . . . 36
4.4 Running the simulator . . . 36
4.4.1 An AI model as a player . . . 36
4.5 Logging and repeatability . . . 37
4.6 Fidelity and the reality gap . . . 37
4.7 History . . . 38
4.8 Summary . . . 38

5 System Architecture and Autonomy **39**
5.1 Architecture . . . 39
5.1.1 Where the control loop belongs . . . 39
5.1.2 Loop timing on the phone . . . 40
5.1.3 Fleet communication . . . 41
5.1.4 Motion authority . . . 41
5.2 Notation and frames . . . 42
5.3 State estimation . . . 42
5.3.1 Validation against motion capture . . . 43
5.4 Occupancy mapping . . . 44
5.4.1 The ground plane . . . 44
5.4.2 Inspecting the classifier . . . 45
5.4.3 Floor veto from the ARKit mesh . . . 45
5.4.4 Glass . . . 46
5.5 Vehicle model and velocity control . . . 46
5.5.1 Kinematics . . . 46
5.5.2 Velocity controller . . . 46
5.6 Turning policy . . . 47
5.7 Local planner . . . 48
5.7.1 Dynamic window . . . 48
5.7.2 Trajectory rollout with braking . . . 49
5.7.3 Admissibility . . . 49
5.7.4 Selection . . . 49
5.8 Control laws for each autonomy level . . . 49
5.8.1 L0: teleoperation . . . 50
5.8.2 L1: assisted teleoperation . . . 50
5.8.3 Pursuit toward a goal . . . 51
5.8.4 L2: direct waypoint . . . 51
5.8.5 L3: waypoint with obstacle avoidance . . . 51
5.8.6 L4: frontier exploration . . . 51
5.8.7 L4: target search . . . 52
5.8.8 L5: task autonomy . . . 52
5.9 Outdoor guidance and heading fusion . . . 54
5.9.1 Geographic goals . . . 54
5.9.2 Heading fusion . . . 54
5.10 Multi-rover map fusion (prototype) . . . 55
5.10.1 Inputs and accumulation . . . 55
5.10.2 Registration . . . 56
5.10.3 Scoring and acceptance . . . 57
5.10.4 Fusion . . . 57
5.10.5 Validation so far . . . 57
5.10.6 Limitations . . . 58

5.11 Summary 59

Introduction

Robots in real missions

Robots have been part of disaster response for more than two decades, and the same problems keep returning. At the World Trade Center in 2001, every robot on the pile was teleoperated. The one robot that was lost and not recovered was a wireless unit: it lost its control link and had no onboard intelligence to move and reacquire the signal [23]. At Fukushima Daiichi in 2011, the concrete walls of the reactor buildings were expected to block radio, so the Quince robot was driven over a cable, and teleoperation had to be simple enough for first responders rather than the researchers who built it [26]. At the DARPA Subterranean Challenge Final in 2021, each team was allowed a single Human Supervisor to interact with its robots, and one operator managed eleven robots in the Louisville Mega Cavern [10, 2]. At the Champlain Towers South collapse in Surfside, Florida, the same year, drones flew 304 missions for mapping and strategic operations alone, and the drone teams coordinated through a WhatsApp group chat [25].

Read together, these deployments show three recurring failures. Communication fails, and a robot that depends entirely on its link stops being useful the moment the link drops. One person ends up juggling several robots with no shared picture of where they are and what they need. The robots carry too little autonomy for the job, or an amount chosen by trial and error rather than by measurement. SubT showed that one-person supervision of a fleet is possible. It also showed its limits: the operator who ran eleven robots described them as "quite a lot to be managed by a single human operator under a constrained time limit," and operators across teams were still the bottleneck for confirming finds, retasking robots, and freeing stuck ones [2].

One operator, many robots

This thesis studies the setting those missions point toward: one human operator supervising a fleet of ground robots. Three questions make that setting hard.

1. **Autonomy.** What is the minimum level of autonomy each robot needs to complete a mission, and when should the human step in?

2. **Attention.** When several vehicles need the operator at the same time, how should the operator's attention be scheduled across them?

3. **Bandwidth.** How much telemetry can each robot send as the fleet grows, and what should the operator see continuously versus on demand?

These questions interact. More onboard autonomy means each robot asks for help less often and can survive a lost link, which frees operator attention and reduces the bandwidth each robot needs. More autonomy also costs development effort, compute on the vehicle, and operator trust, and a robot that is more autonomous than the mission needs is harder to understand when it misbehaves. The practical goal is the lowest level of autonomy that finishes the job and survives a lost link, with a human takeover always one action away.

Levels of autonomy

To make the autonomy question measurable, this thesis uses a six-level ladder for a single ground vehicle (Table 1). Each level adds one responsibility to the robot and removes one from the human.

Table 1: Autonomy levels used in this thesis. Runtime names are the values the ARGOS operator application publishes to Terra.

Level	Name	Robot does	Runtime name
L0	Full teleoperation	Nothing on its own; the human controls all motion.	`teleop`
L1	Assisted teleoperation	The human drives; the rover prevents collisions.	`assisted_teleop`
L2	Waypoint, no avoidance	Drives from A to B along predefined waypoints.	`waypoint_direct`
L3	Waypoint, with avoidance	Navigates from A to B and stops for or avoids obstacles.	`waypoint`
L4	Explore and search	Explores an area autonomously and confirms targets.	`explore`, `target_search`
L5	Decision making	Takes a task in plain language and decides its own next waypoints; fleet-level decisions remain future work.	`task`

The ladder is a working taxonomy for this platform, not a general standard. Chapter 1 places it among existing level-of-autonomy frameworks. An earlier version of this project used a different ladder, with "stop and ask" as its own level; runtime names and research labels are not interchangeable, so every study condition records the exact software revision and behavior used.

The phone as the robot's brain

A fleet is only practical if each vehicle is cheap enough to field, lose, and replace, and simple enough to power, update, and send out again under time pressure. Robots that navigate in clutter usually carry a dedicated onboard computer cabled to a separate sensor suite. That computer makes perception and planning straightforward to host, but it is also a logistics item: bill-of-materials cost, a thermal and power design, spare parts, and a machine image to flash and maintain on every unit.

A current consumer phone already holds what a small rover needs for local navigation: cameras, an inertial measurement unit, radios, a battery, and a processor able to run real-time vision, often with a depth-capable sensor. The premise of this thesis is to make that phone the vehicle's onboard brain. An operator installs an application, mounts the phone on a low-cost differential-drive chassis, and the phone senses, estimates its pose, plans, and commands the wheels. In the current prototype, a Raspberry Pi receives wheel commands from the phone over Bluetooth and drives the motors; the long-term plan replaces it with a low-cost microcontroller attached to the phone over USB.

Research questions

The central question of the thesis is:

> **RQ.** What minimum level of local autonomy allows one human operator to supervise an increasing number of robots effectively?

Three sub-questions, each the subject of one paper, support it.

RQ1 (phone as brain). Can a consumer phone run the sensing and control loop for a low-cost differential-drive rover well enough for waypoint navigation with local obstacle avoidance, without onboard industrial compute?

RQ2 (shared maps). How well can several phone-mounted rovers build and fuse a shared map under limited bandwidth and noisy depth, compared with single-robot mapping?

RQ3 (supervision). What operator interface and attention scheduling let one person supervise N rovers as onboard autonomy increases, and which vehicle should receive human attention first?

The primary experimental factors are fleet size N and autonomy level A.

Approach

The work is built as three systems that share one robotics core.

- **Terra** is the vehicle. A shared Rust core holds state estimation, local occupancy mapping, planning, the control-authority arbiter, and velocity control. The TerraPhone iOS application wraps that core, reads the camera, pose, depth, and IMU through ARKit and Core Motion, and sends wheel effort to the motor controller. Terra keeps motion authority on the vehicle: it checks command freshness and arbitrates between teleoperation, assisted teleoperation, waypoint autonomy, exploration, and target search.
- **ARGOS** (Adaptive Robotic Group Operator System) is the operator layer. It is a native desktop application that shows the fleet on one map with each rover's GPS fix and accuracy, sends autonomy levels, goals, and held teleoperation, and keeps stop and L0 manual control one click away. ARGOS sends intent; it does not stream low-level velocity as its normal command.

- **Zorvane** is the simulator. It runs the same Terra algorithms, so a mission can be supervised in simulation and on real rovers through the same ARGOS interface.

ARGOS communicates with real rovers and with Zorvane over Zenoh, a publish–subscribe protocol [**TODO: cite Zenoh**].

The planned study has operators supervise rovers through ARGOS in real-world missions and in Zorvane, on tasks such as finding a target object, scouting a route, or covering a sector before a deadline. Two conditions frame it. In the *fixed* condition, one autonomy level is locked for the whole run, which gives a baseline per level. In the *free* condition, the operator may switch levels at any time, and the system logs when, why, and in what context; the operator's choices are the core result. Conditions vary communication loss, the number of robots per operator, debris, deadlines, and unplanned events. Measures are mission success, completion time, operator workload, interventions, and time spent at each level. The design is an early proposal and will be made more rigorous before any human-subject study [**TODO: ethics / IRB approval, sample size, counterbalancing, workload instrument**].

Current status

This section states what exists so that later chapters can separate implemented work from planned work. As of October 2026:

- L0–L3 run in simulation, and L3 has been demonstrated on the real rover. L3 obstacle avoidance runs on the phone from live depth, with a local occupancy map and a voxel map, and has been demonstrated indoors against cables, chair legs, carts, and hallway walls [**TODO: report as a qualitative demonstration until measured trials exist**].
- The ARGOS desktop application provides a fleet map, autonomy selection, waypoint goals, teleoperation, emergency stop, and a first-person video panel, and is connected to Zorvane.
- L4 target search exists in Terra and has passed a deterministic simulator replay; physical detection and hardware validation are outstanding.
- The operator pilot study has not been run. No human-subject results exist yet.

Contributions

The planned contributions are:

1. A phone-as-brain ground rover in which a consumer phone is the only onboard computer for sensing, pose estimation, mapping, planning, and control, with a shared Rust core that runs unchanged in simulation.
2. An autonomy ladder for a supervised ground fleet, implemented as selectable runtime levels with a single onboard arbiter for control authority and takeover.

3. ARGOS, a one-operator fleet supervision interface with a shared fleet map, one-click autonomy switching, and one-click takeover.

4. An experimental study of how much autonomy missions need and when operators choose to intervene, as fleet size, communication quality, and environment vary. [**TODO: Fill in once the study is run; do not claim findings before then.**]

Scope

The thesis treats a homogeneous fleet of Terra ground rovers supervised by one operator. Heterogeneous fleets, aerial vehicles, and multiple cooperating operators are outside its scope. The main experimental scope is L0–L3; L4 is evaluated where the platform supports it. L5 exists as a single-rover prototype in which an on-phone language model chooses waypoints for an operator's task (Section 5.8.8); it has not been tested on hardware and is not a study condition, and fleet-level decision making is future work. Disaster response is the motivating application. The same cost and deployability constraints apply to fire and rescue and to surveillance, but this thesis does not evaluate those settings separately.

Thesis outline

[**TODO: Confirm chapter list and numbering against the final structure.**] Chapter 1 reviews robots in disaster response, levels of autonomy, supervisory control of multiple robots, operator interfaces, communication constraints, and low-cost navigation, and identifies the gaps this thesis addresses. Chapter 2 describes the NEXT Competition 2026, the experimental environment built for it, and how that environment maps to disaster response. Chapter 3 describes the physical design of the rover for the competition. Chapter 4 describes the Zorvane simulator, its fidelity, and how it is used. Chapter 5 describes the system architecture and onboard autonomy, with the control equations for each level: the TerraPhone and ARGOS applications, the communication between them, and the autonomy levels from teleoperation through search. Chapter 6 describes the recommendation system that proposes actions to the operator. Chapter 7 presents the ARGOS operator dashboard. Chapter 8 reports the evaluation. Chapter 9 concludes and outlines future work.

Chapter 1

Background and Related Work

This chapter places the thesis in five bodies of work: robots in real disaster response, levels of autonomy, supervisory control of multiple robots, operator interfaces for fleets, and low-cost onboard navigation. It closes with the gaps that motivate the platform and study in later chapters.

1.1 Robots in disaster response

Rescue robotics has a field record that is unusual in robotics: robots have been deployed at real incidents, and the deployments have been documented. Murphy's account of the field gives the broad picture [24]; this section concentrates on four missions whose lessons recur throughout the thesis.

1.1.1 World Trade Center, 2001

The response at the World Trade Center was the first major use of robots for urban search and rescue. All of the robots used were teleoperated. The only robot lost and not recovered was wireless: it lost its control link, and with no onboard intelligence it could not move to reacquire the signal [23]. The lesson for this thesis is that autonomy has to live on the robot as well as in the link. A robot whose only behavior is to wait for commands fails as soon as the commands stop.

1.1.2 Fukushima Daiichi, 2011

After the Fukushima Daiichi accident, the Quince robot was sent into the reactor buildings. Thick concrete walls were expected to block radio, so Quince used a cable link (VDSL) instead of wireless communication [26]. The robot's developers could not operate it on site themselves, so its teleoperation had to be easy for first responders to learn [26]. **[TODO: Add page or section numbers for both claims.]** Two lessons follow. Communication cannot be assumed, and in some environments the choice is between a tether and a robot that can act alone. The operator is often not the expert who built the robot, so the interface has to be simple.

1.1.3 DARPA Subterranean Challenge, 2021

The DARPA Subterranean Challenge (SubT) asked teams to explore, map, and search underground tunnels, urban structures, and caves with robot teams. Its final event took place in September 2021 in the Louisville Mega Cavern [10]. SubT is the closest existing evidence for one-operator fleet supervision under field conditions: each team had a single Human Supervisor, who was the only person allowed to interact with the robots during a run [2]. One operator, Chao Cao, managed eleven robots [2].

Operator interviews after the final show both that this worked and where it strained [2]. Operators were still the bottleneck: they confirmed artifact reports, retasked robots, and got stuck robots free. Team CoSTAR reported that low-level commands were "too difficult for the operator," so over the years of the program the team moved them onto the robots. Operator Marco Tranzatto asked for "a more centralized way to give commands to the robots." Operator Martin Dlouhý observed that "it's obvious to a human when the robot is not doing well," which argues for keeping a fast human takeover. IEEE Spectrum also noted that relying on a skilled, practiced operator suits a competition team but could be impractical for robots deployed operationally [2].

SubT therefore leaves open the question this thesis studies. Teams arrived at their autonomy levels through years of iteration, not through a measured account of how much autonomy a mission needs and when the human should step in. **[TODO: Add SubT team system papers (for example CERBERUS, CoSTAR, CSIRO Data61) for their autonomy and communication architectures.]**

1.1.4 Champlain Towers South, Surfside, 2021

After the Champlain Towers South collapse in June 2021, drones flew 304 missions for strategic operations alone [25]. The drone teams coordinated through a WhatsApp group chat, and mobile apps on the rubble pile showed GPS errors as large as ten meters [25]. The robots themselves worked; what was missing was a shared operational picture of where each vehicle was, how well it was localized, and who was tasking it.

1.1.5 Recurring lessons

Table 1.1 summarizes the failures in these missions and the design response this thesis takes to each. The right-hand column states a design intent, not a claim that ARGOS would have solved these missions.

1.2 Levels of autonomy

Describing how work is split between a human and an automated system by discrete levels has a long history. Sheridan and Verplank proposed a ten-level scale running from a human doing everything to a computer acting entirely on its own [35]. Parasuraman, Sheridan, and Wickens refined this by noting that automation can be applied at different levels to each of four stages: information acquisition, information analysis, decision selection, and action implementation [30].

Table 1.1: Failures in documented robot deployments and the corresponding design intent in this thesis.

Mission	What went wrong	Design intent in this thesis
WTC 2001	Wireless robot lost its link; no onboard autonomy to recover.	Put autonomy on the robot (L1–L3), not only on the link.
Fukushima 2011	Concrete blocked radio; teleoperation had to be easy for responders.	One simple operator application, with L0 manual through L3 waypoint one click apart.
SubT 2021	One supervisor ran eleven robots but remained the bottleneck; autonomy set by trial and error.	Measure how much autonomy a mission needs and when the human should step in; switch levels in one click.
Surfside 2021	304 missions coordinated by group chat; phone GPS errors up to 10 m.	One shared fleet map with each robot, its GPS fix, and its reported accuracy.

Beer, Fisk, and Rogers adapted these ideas to robotics and to the sense–plan–act structure of a robot, and argued that the appropriate level depends on the task and the environment [4]. For road vehicles, SAE J3016 defines six levels of driving automation [32].

The ladder in Table 1 differs from these in two ways. First, it is defined by mission behaviors on one platform (teleoperate, assist, follow waypoints, avoid, search, decide), so that each level is a selectable runtime mode that can be held fixed for an experimental condition. Second, its upper levels move from a single vehicle's behavior to fleet-level decisions, which general automation scales do not address. **[TODO: Compare with ALFUS (Autonomy Levels for Unmanned Systems, NIST) and cite it once the source is confirmed.]**

A fixed level is also not the only option. Work on adjustable, sliding, or mixed-initiative autonomy lets the level change during a mission, by the human or by the system [15]. **[TODO: Add a survey of adjustable / sliding autonomy and mixed-initiative control.]** The free-switching condition of the planned study belongs to this line of work: it treats the operator's choice of level as data.

1.3 Supervising multiple robots

1.3.1 Neglect tolerance and fan-out

Supervisory control describes a human who sets goals and monitors an automated system rather than controlling it continuously [34]. For robot teams, Olsen and colleagues gave this a quantitative form. *Neglect time* is how long a robot can be left alone before its performance drops below an acceptable level, and *interaction time* is how long the human needs to service it [27]. *Fan-out* is the number of robots one person can effectively operate at once, estimated from those two quantities [28]. Crandall and colleagues tested such interaction schemes in multitasking environments [8], and Crandall and Cummings examined which metrics predict supervisory performance with multiple robots [7]. Cummings and Mitchell modeled operator capacity for supervising multiple unmanned aerial vehicles and included the time robots spend waiting for the operator [9].

This literature frames the thesis question directly. Raising the autonomy level should lengthen neglect time and shorten interaction time, which raises fan-out. What it does not

settle is how that relationship looks on a real low-cost platform under communication loss and clutter, and whether operators, when free to choose, pick the level that the model would predict.

1.3.2 Attention, workload, and situation awareness

When several robots need help at once, the operator must decide whom to serve first. Each robot that waits accumulates wait time, and the operator must also switch context between robots. Chen and Barnes review human factors issues in multirobot control, including operator workload, situation awareness, trust, and the effect of automation on both [6]. Situation awareness, in Endsley's definition, covers perceiving the elements of the environment, understanding them, and projecting their future state [13]. Workload is commonly measured with the NASA Task Load Index (NASA-TLX) [17], the instrument currently proposed for this thesis. **[TODO: Add work on attention allocation and scheduling of operator attention across robots (for example queuing models of operator service).]** **[TODO: Add work on interruptions and task switching cost in multi-robot supervision.]**

1.4 Operator interfaces for robot fleets

The deployments above show what operators ask for: one centralized map from which to command robots [2], and a single shared picture in place of chat-based coordination [25]. Interfaces for multi-robot supervision commonly combine a map-centric overview with on-demand detail for a selected robot. Mixed-reality and immersive command interfaces extend this; Milgram and Kishino's reality–virtuality continuum is the standard way to classify them [21]. **[TODO: Replace the placeholder survey of AR/VR command interfaces for multi-robot supervision (`chen2018survey` in thesis_outline.tex) with a confirmed source.]** **[TODO: Add examples of fleet supervision interfaces from SubT teams and from multi-UAV ground control stations.]**

1.5 Communication constraints

Every deployment in Section 1.1 met a communication limit: a lost wireless link at WTC, radio blocked by concrete at Fukushima, and underground radio limits throughout SubT. Two design responses follow. The first is onboard autonomy that keeps the robot safe and useful when the link drops. The second is a telemetry design that sends low-bandwidth state continuously (pose, mission state, health, autonomy level, and whether the robot needs help) and sends high-bandwidth data such as video, point clouds, and maps only when the operator asks for it. **[TODO: Cite communication-aware autonomy and multi-robot networking work, including SubT communication architectures (mesh radios, deployable relay nodes).]** **[TODO: Cite work on bandwidth scaling of telemetry with fleet size.]**

1.6 Low-cost onboard navigation

1.6.1 Local planning

Local obstacle avoidance for differential-drive robots is well established. The dynamic window approach searches the velocities a robot can reach within one control cycle, discards those that lead to collision, and scores the rest on heading toward the goal, clearance, and forward speed [14]. Because it replans every cycle, a sequence of obstacles updates the trajectory without a single precomputed path. **[TODO: Add other local planners used for comparison (for example timed elastic bands, vector field histogram), if the methods chapter uses them.]**

1.6.2 Phones as robot computers

Phones have been used in robotics as teleoperation clients, communication links, and extra cameras. **[TODO: Cite prior smartphone-based robots and phone-as-controller platforms.]** Using the phone as the vehicle's only onboard computer depends on how well it estimates its own motion. Visual–inertial odometry (VIO) fuses camera and IMU data for pose; Delmerico and Scaramuzza benchmarked monocular VIO algorithms on flying-robot hardware [11]. On iOS, ARKit provides camera pose and, on supported devices, depth. Kim et al. compared four off-the-shelf proprietary VIO systems (Apple ARKit on an iPhone 12 Pro Max, Google ARCore, the Intel RealSense T265, and the Stereolabs ZED 2) on indoor and outdoor sequences, including a vehicle drive of about 3 km, and found ARKit the most stable and accurate of the four [19]. This result supports using the iPhone's ARKit pose as the rover's primary localization source. **[TODO: Add the second phone VIO reference from the literature review (Marino 2022) with full details.]** On Android, ARCore provides motion tracking that combines visual feature points with the IMU, and depth maps from the main RGB camera on supported devices [16]. A single Rust library can serve both platforms through UniFFI, which generates Swift and Kotlin bindings [22].

1.7 Multi-robot coordination

At the upper end of the autonomy ladder, robots coordinate among themselves. ALLIANCE is a behavior-based architecture for fault-tolerant cooperation, in which robots take over tasks that teammates fail to complete [31]. Market-based approaches allocate tasks through auctions and bids among robots [12]. Collaborative mapping across several robots is the subject of Paper 2 of this thesis. **[TODO: Add collaborative / multi-robot SLAM references for Paper 2.]**

1.8 Gaps addressed by this thesis

The work reviewed above leaves four gaps that this thesis targets.

1. **Measured autonomy requirements.** Field teams, including at SubT, chose autonomy levels through iteration. Fan-out models predict the effect of autonomy on supervision, but

there is little measurement of the minimum level real missions need, or of when operators choose to step in when they may switch freely. [**TODO: Confirm this claim against the literature before submission.**]

2. **Cost and deployability.** Platforms with dedicated onboard computers are expensive to field in numbers. A phone-as-brain rover tests whether commodity hardware can carry L0–L3 autonomy.

3. **An integrated system.** Onboard avoidance, an onboard authority arbiter, and the operator interface are usually studied separately. This thesis builds them as one system, so that a change of level in the interface maps directly to a change of behavior on the robot, in simulation and on hardware.

4. **A simple shared operator picture.** Field operators asked for one centralized map and simple controls. ARGOS provides a single fleet map with one-click autonomy switching and takeover, designed for operators who are not the robot's developers.

The next chapter describes the competition that provides the study's mission, and the environment built to run it.

Chapter 2

The NEXT Competition and the Experimental Environment

Chapter 1 ended with four gaps, the first of which is that the autonomy a mission needs is rarely measured. Measuring it needs a mission: a task with a clear goal, a deadline, a fixed arena, and a score that does not depend on who is judging. This thesis uses the NEXT Competition 2026 for that purpose. This chapter describes the competition (Section 2.2), the hardware, software, and network environment built to take part in it and to run the study on it (Section 2.3), and how that environment stands in for the disaster-response setting that motivates the thesis (Section 2.4). Section 2.5 then places the study conditions from the Introduction on the NEXT field.

2.1 Why a competition

The deployments reviewed in Chapter 1 are the motivation, but none of them can be repeated. A collapsed building does not wait for a second trial, and the conditions in one run cannot be held fixed for the next. A competition is the opposite: the field, the objects, the time limit, and the scoring are written down in advance and are the same for every run. That makes it a practical middle ground between a laboratory obstacle course, which is repeatable but has no mission, and a real incident, which has a mission but cannot be repeated. The DARPA Subterranean Challenge played the same role for underground search [10], and its single-supervisor rule is the closest precedent for the one-operator question this thesis asks [2].

NEXT is smaller and cheaper than SubT, and it is held outdoors on a sports field. Those two facts matter for this thesis: a low-cost phone-based rover is a realistic entry, and the field is large enough that GPS, a long-range network link, and operator attention all become constraints, while still being safe to test on.

2.2 The NEXT Competition 2026

NEXT is a two-alliance robot competition played on Field 3, West Campus, George Mason University [1]. **[TODO: Name the organizer, the year the competition started, and the date of the 2026 event, from the published rules or the NEXT project notes.]**

The project's simulator, Zorvane, encodes the *NEXT Competition 2026 Rules* dated August 7, 2026, and the summary below follows that encoding rule by rule. It is not a substitute for the rules document, which should be cited directly once it is in hand.

2.2.1 Match format

Two alliances, Red and Blue, play one 90-minute match on a regulation soccer field and swap sides at halftime. Each alliance commands its vehicles from its Base, a 20 × 10 ft canopy tent behind its end line (Rule 2.2). The Base is the only place a person may touch a vehicle. Everything that happens on the field during the match is therefore done by the robots, under remote command from the tent.

2.2.2 Field and game objects

The field is a regulation soccer pitch of approximately 110 × 70 yards, which the simulator takes as 100 × 64 m (Rule 2.1). At the start, 99 tennis balls lie in each penalty box (Rule 3.1). Twenty 5-gallon buckets stand on the field: ten orange and ten blue, one colour per half, each either at ground level or raised to 1 m (Rule 3.2). Red scores in the orange buckets, which stand on Blue's half; Blue scores in the blue buckets on Red's half (Rule 3.2b). To score, a team must therefore collect balls near its own end and carry or launch them into the opponent's half.

2.2.3 Scoring and penalties

Table 2.1 summarizes the scoring rules as encoded. A ball in a ground-level bucket scores one point, or two if the bucket is within 30 m of the opposing Base. A raised bucket scores three or six points on the same distance rule. A vehicle hit by a ball launched by the other alliance is *tagged* and frozen for fifteen minutes (Rule 4.4). Every ball sent out of bounds and every bucket knocked over costs seven points (Rules 4.3 and 5.6).

2.2.4 What the competition asks of a robot

Read as a robotics problem, a NEXT match is a loop of five tasks: *search* the field for balls, *collect* them, *return* to a loading point, *travel* to a target in the far half, and *deliver* with a precise final approach. Each of those tasks fits a different rung of the autonomy ladder in Table 1. Searching a penalty box for balls is a coverage problem (L4). Travel across the pitch is a long waypoint drive with occasional obstacles, such as other robots and buckets (L2–L3). The final approach to a bucket rim is a short, precise manoeuvre where a mistake costs seven points (L0–L1, or an L3 approach the operator watches closely). Because one alliance's operators are confined to the Base and may run several vehicles, the match is also a direct test of the one-operator, many-robots setting.

2.3 The experimental environment

This section describes the environment as it exists in the repositories in October 2026. The same environment serves two purposes: it is the team's competition stack, and it is the instrument

Table 2.1: NEXT Competition 2026 rules as encoded in the Zorvane simulator. Rule numbers are those cited in the simulator source. [**TODO: check every row against the published rules**]

Rule	Content	Consequence for the rover
2.1	Regulation soccer field, about 100 × 64 m	Distances of tens of metres; a straight run across the pitch is 100 m.
2.2	Each alliance has a Base tent; a person may touch a vehicle only at the Base	All field work is remote; a stuck or faulted rover is lost until it returns or the match ends.
3.1	99 tennis balls in each penalty box	Collection is a search-and-gather task, not a single pickup.
3.2	10 orange and 10 blue 5-gallon buckets, ground level or raised to 1 m	Delivery targets are spread over the opponent's half.
4	Ground bucket 1 point (2 within 30 m of the opposing Base); raised bucket 3 (6)	Deeper delivery pays more, and needs a longer trip.
4.3, 5.6	−7 per ball out of bounds and per bucket knocked over	Collisions with targets are costly; precise final approach matters.
4.4	A vehicle hit by an opposing launched ball is frozen for 15 minutes	A rover can be disabled for a sixth of the match by an outside event.
–	One 90-minute match, sides swapped at halftime	Endurance and battery life for the whole match.

for the thesis study. Figure 2.1 shows its parts.

2.3.1 The router

All fleet traffic passes through one Zenoh router running on a small cloud virtual machine. Zenoh is a publish–subscribe protocol [**TODO: cite Zenoh**]; every rover, the ARGOS desktop application, and the Zorvane simulator connect *out* to the router over TLS and authenticate with a fleet account. All topics share one prefix, and each rover publishes and subscribes under its own numeric rover ID. Because every party dials out, neither the operator's laptop nor the phone needs a public address or an open port, which is what lets a phone on a field connect to an operator in a tent without local network setup. The same machine hosts an independent observability collector that stores timestamped diagnostic events from the phone, the operator station, and the simulator, and exports them for replay.

The router is not part of the control loop. It carries intent (levels, goals, held teleoperation, stop) and telemetry (pose, status, occupancy, video signalling), but every motion decision is checked again on the vehicle. Fleet membership allows publishing and subscribing; it does not give per-operator roles, and the vehicle's own runtime checks remain the final command boundary. [**TODO: State whether the phone reaches the router over cellular data or over field Wi-Fi in the NEXT setup, and measure the round-trip latency from Field 3.**]

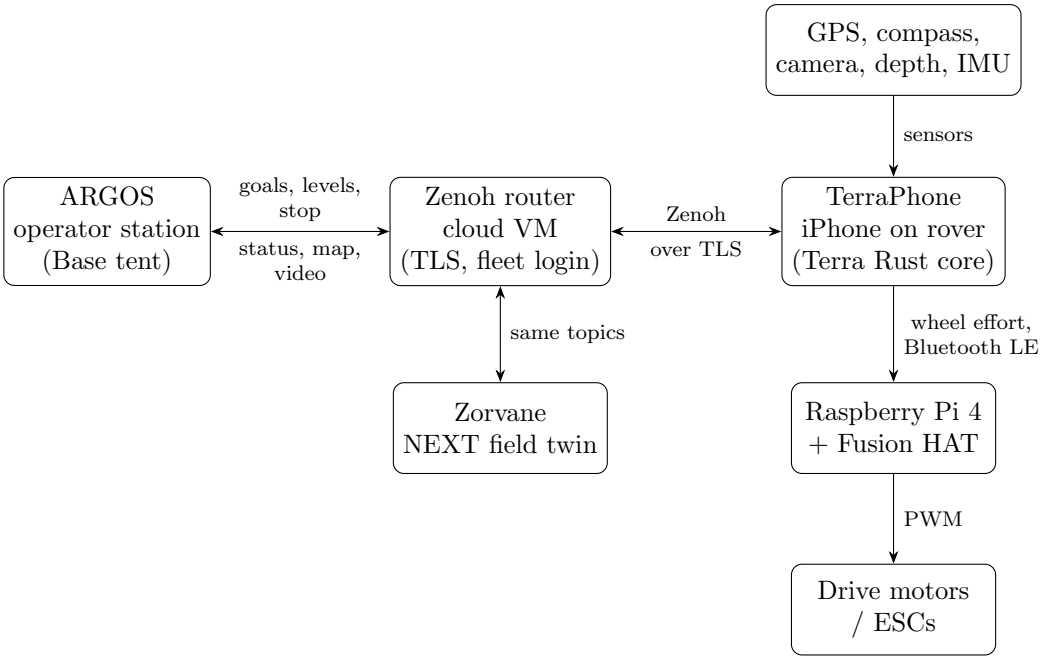


Figure 2.1: The experimental environment. The operator station and the rover never talk directly: both connect out to one authenticated Zenoh router. Zorvane speaks the same topics as a real rover, so the operator station cannot tell them apart. Planning and control run on the phone; the Pi only applies wheel effort.

2.3.2 The phone as the brain

An iPhone rides on the rover and runs TerraPhone, a Swift application around the shared Terra Rust core. The phone provides every sensor the autonomy uses: the rear camera and ARKit visual–inertial pose, depth where the handset has it, the Core Motion IMU, GPS, and the compass. The Rust core holds state estimation, a rolling local occupancy map (200×200 cells at 0.1 m, so 20×20 m), the control-authority arbiter, the planners, and velocity control. The control loop ticks every 10 ms, and wheel commands go out every fifth tick, at 20 Hz. The arbiter is the single owner of motion: teleoperation, assisted teleoperation, the two waypoint levels, exploration, target search, and task autonomy all pass through it, and an emergency stop latches until an explicit, healthy reset.

The phone does not run a second planner when it supervises the simulator; in that mode it sends goals and control intent to Zorvane instead. Depthless handsets hold any level that needs a map.

2.3.3 The Pi motor bridge

The phone cannot drive motors directly, so a Raspberry Pi 4 with a SunFounder Fusion HAT acts as a motor bridge. The phone sends normalized wheel effort to the Pi over Bluetooth Low Energy; the Pi applies it to motor ports or to PWM outputs for electronic speed controllers (ESCs). The Pi runs no autonomy and none of the Rust crates. It holds each drive command for 250 ms and then sets every output to its safe value, and an external battery cutoff removes motor power independently of both computers. Zero effort coasts; it does not brake. Section 3.5

describes how the Pi's own safety duties changed in October 2026. Chapter 3 describes the hardware in detail, including the planned replacement of the Pi by a microcontroller attached to the phone over USB.

2.3.4 The operator station

The operator runs the ARGOS desktop application, written in Rust with the GPUI toolkit, on a laptop. For NEXT the laptop sits in the alliance's Base, which is where the rules allow the team to be. ARGOS shows each rover on one map over satellite imagery with its GPS fix and accuracy, a local occupancy map, first-person video over WebRTC, and the autonomy controls. The operator selects Local or Outdoor GPS navigation and an autonomy level, sends goals by clicking the map, arms or disarms the hardware, and has stop and L0 manual control one click away. ARGOS binds every non-neutral command to the rover's current run and authority revision, so a stale command from before a takeover or a reset is rejected by the vehicle rather than acted on.

Several changes in late October 2026 shaped how the operator works with it. A goal is set with a single click on the map, which opens a confirm popup anchored to that point; nothing is sent until the operator presses Send goal, and a drag still pans the map. The local occupancy grid is drawn on the main map under the rover, in the same frame as its goals, so a rover placed by GPS can also take local goals by a click on its grid; the separate local-map tab was removed. When manual driving is refused, ARGOS names the failing check (no status yet, an unanswered command, stale status and its age, an active stop, a Terra hold with its reason, or a non-manual level) instead of one generic error. The navigation panel shows the reported heading uncertainty and its age against the outdoor limits. A Sim/Real toggle reconnects the whole dashboard to the simulator or to the real fleet at runtime, so one session can switch between Zorvane and a rover. Capture 3D and Live 3D show the rover's labelled depth points, which is how the operator can see what the mapper thinks is ground and obstacle (Section 5.4.2). A Task panel sends an L5 task and shows the model's plan, questions and recent steps (Section 5.8.8). ARGOS has a user documentation site and downloadable builds for macOS, Windows and Linux, produced from a version tag; the first release is v0.1.0.

2.3.5 Outdoor GPS navigation and heading

On a 100 m field the phone's visual–inertial pose alone is not enough to reach a bucket on the other half: it drifts, and it has no notion of where the buckets are in the world. Terra therefore has an outdoor GPS mode, in which a geographic goal (latitude and longitude) is converted, at most twice per second, into a local target up to 3 m ahead. The local planner and the L3 footprint and braking checks still choose every command, and GPS never modifies the AR pose or the occupancy map. Forward speed in this mode is capped at 0.3 m/s.

A mission is admitted only at L3, with armed hardware, fresh visual–inertial feedback, a healthy local map, a GPS fix no older than 3 s with accuracy of 10 m or better (and no worse than the arrival radius), and a heading no older than 2 s with accuracy of 30° or better. The default arrival radius is 5 m, configurable from 2 to 20 m, and arrival is confirmed by three distinct fixes inside the radius. Any loss of GPS, heading, tracking, map, arming, or authority

latches a pause; recovery alone does not resume, and the operator must press Resume in ARGOS. ARGOS independently holds an outdoor mission when the reported heading uncertainty exceeds $\pm 35°$.

Heading turned out to be the weak input. The phone's magnetometer, mounted close to the drive motors, read headings with errors of 30–50°, so a compass-only heading tripped the uncertainty hold often. Since October 2026, Terra fuses three sources: the ARKit yaw carries the heading between corrections, the GPS course over ground (used only at 0.5 m/s or faster and while turning less than 10°) gives a tight lock, and the compass only seeds the estimate and never claims to be better than its own reported accuracy. In practice, the rover is driven in a straight line for a few seconds at the start of an outdoor run to lock its heading. **[TODO: Measure heading error before and after the fusion on Field 3 and report it in the evaluation chapter.]**

2.3.6 Frontier exploration

Searching a penalty box, or a sector of a disaster site, is a coverage task, and Terra implements it as time-bounded frontier exploration. A frontier is a free map cell next to unknown space. In the autonomous `explore` level, the rover repeatedly picks a frontier, scored by expected information gain, and drives to it through the same local planner and safety arbiter as every other level; there is no per-goal approval. The run ends when its time budget expires, when no reachable frontier remains, when the operator stops or takes over, or when the map goes stale (older than 0.5 s) or a safety hold fires. A blocked frontier is retried and, after three failures, excluded for the rest of the run. Exploration reports elapsed and remaining time, the current target, and the observed free area, and logs each frontier choice.

In the ladder of Table 1, exploration and target search together make up L4: exploration covers ground, target search confirms objects in it. ARGOS offers the two as separate controls, "L4 Explore" and "Search"; both are reported as L4. Above them, the L5 `task` level lets an on-phone language model choose waypoints for a task typed by the operator; it is implemented but has not yet run on a phone, and it is not part of the pilot study (Section 5.8.8).

2.3.7 Zorvane: the NEXT field twin

Zorvane is the project's simulator, built on the Bevy engine with Avian physics, and since October 2026 its only world is the NEXT field. It places a 100×64 m pitch on Field 3 at the field's mapped centroid and heading, taken from OpenStreetMap [29], with an ellipsoid height derived from USGS elevation and the NOAA GEOID18 offset. Because of that placement, the simulator publishes a geographic localization report and simulated GPS on the same topics a real phone uses, so ARGOS shows the simulated rover on Field 3 and Terra's outdoor GPS navigation runs on the simulated field unchanged. The surroundings (turf, fence, light towers, scoreboard, the two Base tents, and nearby buildings) are generated from OpenStreetMap footprints and given colliders. Lighting can be set to noon, overcast, or evening for detector testing.

The game objects are modelled at regulation size: tennis balls of 6.70 cm diameter and 58 g, inside the ITF tolerances [18], and 5-gallon buckets from a product drawing (0.305 m top

diameter, 0.340 m high), ten per half, four of them on 1 m stands, placed by a fixed random seed. The default rover is CrabTrapBot, the team's NEXT robot (Chapter 3). A full scoring loop is implemented: the rover collects balls through its intake, returns to its own goal line, where collected balls become waiting, loads them one at a time into a two-ball dumper on operator command, parks with the outlet over a ground-level bucket of its alliance's colour, and dumps. Balls fall under physics, and only balls that land in the bucket score, with the 1- and 2-point rule. Launching, tagging, out-of-bounds penalties, and knocked-bucket penalties are explained in an intro cutscene but not enforced, so raised buckets are out of reach in simulation.

Earlier versions of Zorvane also held the reference disaster-search mission used to develop the autonomy levels, which was removed when the simulator became NEXT-only on October 24, 2026 (Chapter 4).

2.3.8 Logging and measurement

Every part of the environment records what it did. Terra and Zorvane write a JSONL run log per run containing the mission definition, planner configuration, run identity, every control tick's output, authority requests and changes, mission progress, observed free area, contacts, and near-miss episodes (observed clearance below 0.3 m). ARGOS exports operator session logs whose action tokens and teleoperation sequence numbers join to the rover's receipts, so one intervention is not counted twice. An `experiment/status` topic pairs each rover's run clock with UTC, and clock offsets are kept rather than assumed away. A trial is labelled as *fixed* (one assigned level) or *adaptive* (operator-chosen), and the assigned condition is recorded separately from the levels actually requested and in effect. Truncated recordings are not counted as complete runs, and interaction counts are recorded as workload proxies, not as measured workload.

The phone also keeps an observability journal of sensor, estimator and controller samples, copied off the device after a session; its sample rate is configurable and defaults to 10 Hz. For ground truth on motion, the environment can include an OptiTrack motion-capture system: a small bridge republishes its rigid-body pose on the fleet router, so the same laptop that drives the rover records both the rover's own pose and the true one on one clock (Chapter 5).

2.4 From the field to the real situation

NEXT is a game, and a soccer field is not a collapsed building. The case for using it rests on a structural match: the problems the field forces on a team are the same problems the deployments in Chapter 1 ran into, in a form that can be repeated and scored. The ARGOS design notes describe the intended thesis experiment as "real-world, NEXT-style search and rescue," with simulation used for algorithm development only. Table 2.2 lays out the correspondence.

2.4.1 The lessons from Chapter 1, on the field

Each recurring lesson from real missions has a concrete counterpart in this environment.

- **Lost link (WTC 2001).** The wireless robot lost at the World Trade Center had no onboard autonomy to recover its link [23]. In this environment the rover connects to a

Table 2.2: How the NEXT environment maps to disaster response, and what each element lets the study measure.

In NEXT	In a disaster response	What it tests here
Base tent; vehicles may be touched only there	Command post at the edge of an unsafe zone; responders cannot walk into the hazard to fix a robot (SubT's single Human Supervisor)	Fully remote supervision; a stuck rover is lost until recovered remotely
Tennis balls spread in the penalty boxes	Victims, objects, or areas to be found and reported	Search and coverage (L4), detection, and confirmation by the operator
Buckets in the far half	Delivery or reporting points: casualty collection, supply drop, sensor placement	Long waypoint travel (L2–L3) and a precise final approach
Distance bonus near the opposing Base	Deeper, riskier targets are worth more	Trade-off between reward and the risk of travelling far from the operator
90-minute match	Time-critical response window	Mission success under a deadline; battery endurance
Tagged and frozen for 15 minutes	A robot disabled by debris, a fall, or a lost link	Fleet degradation and operator re-planning around a lost unit
-7 for a knocked bucket or a ball out of bounds	Collateral damage and unsafe actions	Safety cost of autonomy errors; when the operator chooses to take over
100 m outdoor field, GPS, a remote link	Large outdoor sites; phone GPS errors of up to 10 m at Surfside	GPS accuracy gates, heading fusion, and continuing an accepted mission through a link outage
Several robots per alliance, one Base	One operator, many robots	Fleet size per operator and attention scheduling

cloud router over a link that can drop. Terra's design keeps an accepted L3 waypoint mission running locally across a router outage, while expiring manual intent within its 500 ms lease and never re-arming or resuming on reconnection alone; simulated 120-second router outages were used to check this. On the field this can be tested by cutting the router link mid-mission.

- **Simple control for responders (Fukushima 2011).** Quince had to be easy for first responders to drive [26]. ARGOS keeps every level one click away, and the study is designed for operators who did not build the rover.

- **One supervisor as the bottleneck (SubT 2021).** NEXT's Base rule recreates SubT's single-supervisor constraint [2] at a smaller scale. The ball-collection loop produces a steady stream of moments that need the operator, such as loading at the goal, approving a dump, freeing a stuck rover, which is the attention load the study measures.

- **Shared picture and GPS error (Surfside 2021).** Drone teams at Surfside coordinated through a chat group and saw phone GPS errors of up to ten metres [25]. ARGOS shows every rover on one map with its fix and accuracy, and Terra refuses outdoor missions when the fix is worse than 10 m.

2.4.2 Where the analogy breaks

The mapping has limits, and the thesis does not claim more than it supports. The field is flat synthetic turf: there is no rubble, no stairs, no dust, and no confined space, so mobility and terrain are not tested. A tennis ball is not a person: detecting an optic-yellow ball on green turf is far easier than finding a victim, and the simulator's survivor rule is not a sensor model. The network fails because of distance and congestion, not because concrete blocks the radio, and the field has open sky for GPS that a building interior would not. The opposing alliance is an adversary that a disaster site does not have, although tagging is a reasonable stand-in for losing robots to events outside the operator's control. Finally, the competition rewards points, and a response rewards lives and information; the study therefore reports mission measures (success, time, interventions, time at each level) rather than treating the match score as the result. [**TODO: Decide whether the study runs during the actual NEXT match, in practice sessions on Field 3 with the same rules, or both, and state which results come from which.**]

2.5 Study conditions on the NEXT field

The Introduction proposed a study with a *fixed* condition, in which one autonomy level is locked for a run, and a *free* condition, in which the operator switches levels at will and the system logs when, why, and in what context. On the NEXT field, the deck's three mission types become concrete tasks: *find a target* becomes locating balls or a specific bucket; *scout a route* becomes reaching the far half; and *cover a sector* becomes a time-bounded exploration of one penalty box. The conditions the Introduction lists can each be produced on the field: communication loss by cutting the router link, fleet size by the number of rovers per operator, debris by placing extra obstacles, deadlines by the match clock or a shorter budget, and surprises by tagging or by moving a bucket. The same tasks can be run first in Zorvane, against the same ARGOS interface, and then on Field 3.

[**TODO: Fix the task list, the number of rovers, the trial order and counterbalancing, the workload instrument, and ethics approval before running any human-subject trial.**] [**TODO: No trials have been run. Report results only in the evaluation chapter.**]

2.6 Summary

The NEXT Competition 2026 gives this thesis a mission with fixed rules, a deadline, a large outdoor field, and a rule that confines the team to its Base. The environment built for it connects an operator station and phone-brained rovers through one authenticated cloud router, keeps motion authority on the phone, drives the motors through a Pi bridge, navigates outdoors on GPS with a fused heading, explores frontiers autonomously, and has a simulated twin of the same field on the same topics. The structure of a NEXT match maps onto the problems seen in real disaster response, communication loss, a single supervisor, too little or too much autonomy, and the need for one shared picture, while the field leaves out terrain and real victims. The

next chapter describes the rover that has to play it.

Chapter 3

Rover Physical Design

Chapter 2 described the NEXT field and the environment around the rover. This chapter describes the rover itself: what the rules ask of its body, the two chassis the project uses, the ball-handling mechanism, the phone mount, the drive electronics, and the planned replacement of the motor bridge. The guiding principle is the one from the Introduction: the phone is the robot's only computer, so the rest of the vehicle should be as simple and cheap as the task allows.

All dimensions below are taken from the CAD models as the Zorvane simulator reads them. They describe the design, not the built robot, and Section 3.8 lists what still has to be measured.

3.1 Design requirements from the rules

Each rule in Table 2.1 turns into a physical requirement.

Ball handling. The game object is a regulation tennis ball, about 6.7 cm in diameter and 58 g [18], and 99 of them start in each penalty box. The rover needs an intake that gathers balls from the turf without stopping for each one, a store to carry several, and a way to put them into a bucket.

Bucket height. A ground-level 5-gallon bucket has its rim about 0.34 m above the turf; a raised bucket's rim is about 1.34 m up. Dumping into a ground bucket needs an outlet above 0.34 m that can reach over the rim; scoring in a raised bucket needs a launcher.

Knocked buckets cost seven points. The front of the rover must be able to park against a bucket to dump without pushing it over or swallowing it into the intake.

Distance and time. The field is 100 m long and the match lasts 90 minutes. At the outdoor GPS mode's 0.3 m/s cap, a single crossing of the pitch takes about five and a half minutes, so drive speed, battery life, and traction on synthetic turf all matter.

No touching outside the Base. Every recovery from a fault has to be remote. The drive electronics must fail safe on their own, and the rover must not need a person to reset it after a lost link.

Tagging. A vehicle hit by a launched ball is frozen for 15 minutes. The body cannot prevent this, but a smaller profile is a smaller target.

3.2 Two chassis

The project uses two bodies with the same software. Both are differential-drive vehicles, so Terra's controllers, planners, and arbiter run on either; only the geometry, the wheel layout, and the actuator mapping change. Table 3.1 compares them.

Table 3.1: The two rover bodies, from their CAD models as used by the simulator. [**TODO: replace with measured values from the built robots**]

	Terra development chassis	CrabTrapBot (NEXT rover)
Role	Development and indoor obstacle-avoidance tests	Competition robot for NEXT
Size (W × H × L)	0.403 × 0.144 × 0.384 m	0.607 × 0.614 × 0.601 m
Wheels	Four, radius 45 mm	Four goBILDA 144 mm wheels (radius 72 mm)
Track width	0.350 m	0.526 m between tyre centres
Wheelbase	0.240 m	[**TODO: measure**]
Steering	Differential, skid-steer	Differential, four-wheel skid-steer
Motor outputs	Four DC motors on Fusion HAT motor ports (`terra-mini` layout)	ESCs on Fusion HAT servo (PWM) pins
Ball handling	None	Intake, internal store, top dumper
Phone mount	Portrait, top edge up, rear camera forward	Landscape, rear camera forward
Mass	[**TODO: measure; simulator uses 20 kg**]	[**TODO: measure; simulator uses 20 kg**]

3.2.1 The Terra development chassis

The development chassis is the small rover used for the indoor L3 obstacle-avoidance demonstrations in the Introduction. Its CAD model is published in centimetres; reading those units as inches had once made the simulated rover 2.54 times too large, and the model is now scaled from its own unit metadata. Its four DC motors connect to the Fusion HAT motor ports, two per side, and the committed `terra-mini` layout routes left effort to ports M2 and M3 and right effort to M0 and M1, each limited to ±0.4 effort and half power until the drive is calibrated.

3.2.2 CrabTrapBot

CrabTrapBot is the team's NEXT robot, designed in Fusion and modelled in the simulator from its STEP export. It is a roughly 0.6 m cube on four goBILDA 144 mm wheels, with a low, open-front intake and a dumper on top. Its four wheels skid-steer, so the rover turns by dragging its wheels sideways, and its effective turning track on turf is wider than the 0.526 m geometric track. Its propulsion is driven through ESCs on the Fusion HAT's PWM pins rather than through the HAT's motor ports. NEXT robots have historically combined an intake with

a shooter for balls (Terra issue #21); CrabTrapBot's design as modelled has the intake and a gravity dumper, and no launcher. [**TODO: Describe the built robot: frame material, motors and gearboxes, ESC model, and whether a launcher is planned for raised buckets.**]

3.3 Ball handling

CrabTrapBot's name describes its intake. Balls enter through a low opening across the front, past inward-swinging flaps that let a ball in but not back out, like the funnel of a crab trap. Inside, balls collect in a holding area. In the simulator the store holds up to eight balls; when it is full the flaps stay shut and a front bar closes the chassis, so further balls stay on the field rather than jamming the intake. [**TODO: Confirm the real store capacity and the flap mechanism (sprung or gravity).**]

An upper front guard sits above the intake opening, at a height that lets balls pass underneath while keeping a bucket outside the chassis. This is the design answer to the seven-point knocked-bucket penalty: the rover can drive its front against a bucket to dump without the bucket entering the intake.

Delivery uses a top-mounted dumper. At the rover's own goal line, collected balls are moved, one per operator command, from the store into the dumper, which holds two balls about 0.6 m above the turf with its outlet just beyond the front of the chassis. The rover then drives into the opponent's half, parks against a ground-level bucket, and dumps when the outlet is within 9 cm of the bucket opening. A 0.6 m outlet clears the 0.34 m rim of a ground bucket but not the 1.34 m rim of a raised one, so raised buckets, worth three or six points, need a launcher that the current design does not have. In the simulator the dump chute is idealized; panel geometry, hinge torque, and ball contact with the panels are not modelled. [**TODO: Measure the real dumper's outlet height and the success rate of a dump into a ground bucket.**]

The load-at-goal step is deliberately an operator action. It gives the operator a defined, repeatable moment of attention per scoring cycle, which is useful for the study: it is a natural place to measure how quickly an operator responds when several rovers reach their goal lines at once.

3.4 The phone mount

The phone is the rover's only sensor head, so its mount sets what the rover sees. On both chassis the rear camera faces forward. On the development chassis the phone stands upright in portrait with its screen facing backward; on CrabTrapBot it is mounted in landscape, rolled $\pm 90°$, and the roll is entered in the app's mounting settings. The software keeps separate transforms from the phone's motion axes and from ARKit's camera axes to the rover body, and the compass reference follows whichever phone edge is most horizontal in the saved mount.

A mount calibration procedure records the camera height above the ground and any roll, pitch, and yaw relative to the rover axes, then fits the camera's forward and left offsets from two slow in-place turn passes with the motors disabled. The mount must be rigid: any play between

phone and chassis appears as pose error. On CrabTrapBot the phone sits in landscape at a camera height of 0.23 m, with its screen toward the rover and the rear camera facing forward. The software's nominal mount is upright portrait, so the landscape mount is only correct once the calibration's roll is set to $-90°$ (top edge of the phone toward the rover's left) or $+90°$ (toward its right); the sign convention has not yet been checked on the rover. Left at roll 0, the calibration produced two faults found in October 2026: the IMU's yaw-rate axis was 90° off, so the estimated yaw rate was wrong (Chapter 5), and the body appeared tilted on its side, so the occupancy map stayed in its fallback horizon band and marked a ring of floor as obstacles. Saving a calibration now stops the controller first, because a save was silently refused while the rover was still in a driving mode (Terra PR #73). [**TODO: Report the measured camera height and offsets for CrabTrapBot, and confirm the sign of the landscape roll.**]

The mount's position also affects the compass. Close to the drive motors, the phone's magnetometer read headings with errors of 30–50°, which is why Terra fuses visual–inertial yaw, GPS course, and the compass rather than trusting the compass (Section 2.3.5). Moving the phone further from the motors is a physical mitigation worth testing alongside the software one. [**TODO: Test heading error at two or more mount heights on CrabTrapBot.**]

3.5 Drive electronics

3.5.1 The Pi motor bridge

Today the phone drives the motors through a Raspberry Pi 4 with a SunFounder Fusion HAT. The phone sends normalized actuator commands over Bluetooth Low Energy; the Pi validates them and writes them to the HAT. The Pi software is a single compiled ARM64 executable, so no Python installation or source checkout is needed on the rover. A configurable layout of up to 16 actuators maps commands to outputs of four kinds: DC motors (signed effort), bidirectional ESCs (reverse, neutral, and forward pulse widths), unidirectional ESCs, and positional servos. ESC and servo outputs use 50 Hz pulses, and the DC motor channels use a separate 100 Hz timer group so that the two do not share a timer. The bridge always offers all of the HAT's channels, the four motor ports M0–M3 and the twelve PWM outputs P0–P11, and the saved layout alone decides which are opened; layout validation rejects two actuators on one pin and mixed timer groups.

Two faults found on the bench in October 2026 are worth recording, because both looked like control problems and were not (Terra PR #69). First, the HAT's kernel driver rejects a duty-cycle write to a disabled channel, and the vendor library disables a channel whenever any stale object for it is destroyed; the DC motor path never re-enabled its channels, so a four-motor rover faulted on every wheel. The bridge now owns its channels' lifetime and re-enables once and retries on that error. Second, the bridge used each drive command for one 50 ms output loop and sent neutral on any loop without a fresh packet, while the phone also sent every 50 ms; Bluetooth jitter therefore alternated drive and stop, and the rover shuddered. The bridge now holds the latest command for 250 ms before going to neutral.

3.5.2 Safety in the hardware path

Because nobody may touch the rover outside the Base, the hardware path must fail safe on its own. In the relay as of late October 2026 the Pi enforces, independently of the phone:

- a 250 ms command lease, after which outputs go to their safe state; an explicit neutral command, a Bluetooth disconnect, a fault or a layout change stop at once;
- a safe state per output kind: zero power for DC motors, a continuous neutral or stop pulse for ESCs (never a zero-width pulse, which some ESCs read as a fault), and a configured safe position for servos;
- an external battery cutoff switch that removes motor power and that the Pi cannot override.

An earlier version of the bridge also kept its own arm state and read a hardware gate input, with a 200 ms watchdog. The relay was simplified on October 23, 2026: the Bluetooth protocol still carries arm and disarm frames, but the relay holds no arm state of its own and reads no gate, and Terra's hardware documentation still describes the older design. [**TODO: Decide where arming is enforced now (phone, Rust core, or Pi), state it here, and bring Terra's hardware documentation in line with the relay.**]

Zero effort coasts rather than brakes. The Raspberry Pi image used on the rover adds a fallback Wi-Fi hotspot and SSH over the USB-C port, so a technician at the Base can reach the Pi without the field network.

3.5.3 Skid-steer turning

A four-wheel skid-steer rover scrubs its tyres whenever it turns, and turning in place on turf is hard on the drivetrain and on the phone's pose estimate. Terra applies a shared turning policy to all managed commands (ARGOS teleoperation and every autonomous level): yaw rate is limited to 0.5 rad/s, turns ramp in and out, the minimum turning radius is 0.5 m, turning in place is off by default, and reverse is limited to 0.15 m/s. These are software starting values, not calibrated limits. [**TODO: Calibrate the turning limits on CrabTrapBot on synthetic turf.**]

3.5.4 Rover profiles

The two chassis share the software but not their drive. The small rover's four DC motors stalled whenever it was driven from ARGOS or by an autonomy level, though they ran from the phone's debug screen: the debug screen sends raw effort, while ARGOS and the levels go through the VIO speed loop, whose gains were tuned for CrabTrapBot, and a 0.2 m/s request became an effort of about 0.13, inside the motors' deadband. Terra now keeps a *rover profile* per body: footprint, track width, planning clearance, speed and turn limits, whether turning in place is allowed, top speed, a skid factor, the wheel deadband and the spin effort (Terra PR #71). The controller gains follow from the profile rather than being set by hand: the linear feedforward is $1/v_{\text{top}}$, the yaw feedforward is $s\,b/(2v_{\text{top}})$ for skid factor s and track b, and the PI gains are

fixed multiples of the feedforward. Table 3.2 lists the two presets; the `terra` preset reproduces the earlier single configuration. The small rover's planning clearance was set to zero because, with 0.1 m of clearance, the A* footprint needed a gap five cells wide and a 0.4 m doorway flickered between passable and blocked (Terra PR #72).

Table 3.2: Rover profile presets in Terra (`RoverProfile`). The `terra_mini` deadband, spin effort and top speed are estimates, not measurements.

Field	`terra`	`terra_mini`
Footprint for planning (m)	0.85×0.85	0.178×0.178
Track width (m)	0.5	0.152
Planning clearance (m)	0.05	0
Speed limit; yaw-rate limit	2 m/s; 2 rad/s	0.4 m/s; 1.5 rad/s
Minimum turning radius (m)	0.5	0.15
Turning in place	off	on
Top speed at full effort (m/s)	3.0	0.8
Skid factor	2.4	2.0
Wheel deadband; spin effort	0; 0.35	0.3; 0.6

The `terra_mini` footprint, 0.178 m square, was measured on the small rover; it does not match the 0.403 m width of the CAD model in Table 3.1. [**TODO: Reconcile the small rover's measured size with its CAD model, and measure its deadband, spin effort and top speed.**]

3.6 Power

The rover carries two energy stores: its drive battery, which also powers the Pi, and the phone's own battery. A 90-minute match sets the endurance target for both. TerraPhone keeps the screen awake while its scene is active, and continuous camera, depth, GPS, and network use draw the phone battery quickly. [**TODO: Specify the drive battery (chemistry, voltage, capacity) and measure drive and phone endurance over a 90-minute run on turf.**]

3.7 Planned: a USB microcontroller in place of the Pi

The Pi is the last general-purpose computer on the rover, and the phone-as-brain argument says it should go. It costs money, needs an operating system image to flash and maintain, boots slowly, and adds a Bluetooth link that can drop. The long-term plan is a cheap microcontroller attached to the phone as a USB peripheral.

The chosen part is an RP2350 that enumerates as a USB network device (a CDC-NCM or CDC-ECM Ethernet gadget) on the iPhone's USB-C port. The phone would then send wheel commands over that wired network link, which avoids the accessory certification that iOS requires for a raw USB serial device. The microcontroller would keep the Pi's safety duties: arming, a watchdog, and safe output states.

Two risks come first. The phone-side risk is whether iOS brings up a generic USB network link from such a device at all; this has not been bench-tested. The network-side risk is that

the board must not advertise a default route or a DNS server, or the phone might send its router traffic over the USB link and lose its connection to ARGOS. [**TODO: Bench-test the RP2350 USB network link on the rover's iPhone before committing the design; report the result here.**]

3.8 Open measurements

Several numbers in this chapter are still design values or simulator placeholders. Before any of them appear in results, they must be measured on the built robot: mass (the simulator uses 20 kg, unsourced), motor and ESC limits and the resulting top speed (the simulator keeps a 2.29 m/s cap carried over from an earlier model), wheelbase, store capacity, dumper outlet height, phone mount offsets, battery specification and endurance, and the turning limits on turf. [**TODO: Add a photo and a labelled drawing of CrabTrapBot.**] [**TODO: Add a bill of materials with costs, to support the low-cost argument.**]

The next chapter describes the simulator in which this rover, and a fleet of copies of it, can be driven on a virtual NEXT field.

Chapter 4

Simulation

Chapter 2 introduced Zorvane as the digital twin of the NEXT field. This chapter describes the simulator itself: what it is for, how it is built, what it models and what it leaves out, and how it is used in this thesis. The guiding rule is that the simulator must be indistinguishable from a real rover *at the interface*: an operator station or a test script connected to Zorvane should not need to know that no hardware exists. Below that interface, the simulator is deliberately simpler than the world, and Section 4.6 lists exactly where.

4.1 Role in the thesis

The simulator has four jobs.

1. **Algorithm development.** The autonomy levels, from assisted teleoperation to frontier exploration, were developed against the simulator before they were run on a phone. Because Zorvane links the same Terra crates the phone uses, a change to a planner is exercised in simulation with no porting step.

2. **Interface development.** ARGOS was built and tested against Zorvane: the fleet map, autonomy controls, waypoint goals, held teleoperation, first-person video, and outdoor GPS missions all work against the simulator before they meet a rover.

3. **Scale.** A team owns a handful of rovers; the simulator can field up to 32. The ARGOS design notes plan to repeat the study measures at 15 to 20 vehicles, a fleet size that is only practical in simulation.

4. **Practice and training.** The NEXT field, its rules, and its scoring loop let a team rehearse a match, and let a study participant learn the operator station, before going to Field 3.

The simulator is *not* the venue for the thesis's main results. The ARGOS design notes state that the thesis experiment is real-world, NEXT-style search and rescue, and that the simulator is for algorithm development. Results obtained in Zorvane are reported as such and are not presented as evidence of physical performance.

4.2 Why a custom simulator

General robotics simulators such as Gazebo [20] and photorealistic ones such as AirSim [33] are the usual choices for ground robots. Zorvane was written instead, in Rust, for three reasons that follow from the rest of the system. First, Terra's autonomy is a Rust library; a Rust simulator links it directly, with no middleware bridge and no second implementation to keep in step. Second, the operator contract is Zenoh topics with specific JSON payloads, run and authority revisions, and WebRTC video; matching that contract exactly was simpler in a simulator built around it than by adapting a general one. Third, the NEXT field is a specific, flat, outdoor place, which needs accurate geography and game logic more than it needs articulated-body dynamics or terrain. [**TODO: Add a short comparison with ROS 2 and Gazebo-based fleet simulators, and say what Zorvane gives up by not using them.**]

4.3 Architecture

Zorvane is built on the Bevy game engine [5] with the Avian physics engine [3]. One Bevy unit is one metre. The repository is a Rust workspace with two members: the simulator binary and a vehicle-body crate. The robotics crates (types, control, waypoint following, mapping, navigation, state estimation, transport, autonomy, and the experiment recorder) stay in the Terra repository, where the phone also compiles them, and Zorvane builds against a pinned Terra revision recorded in the repository. Pinning keeps the phone and the simulator on the same autonomy code, and makes every simulated run traceable to an exact revision.

Figure 4.1 shows the main parts.

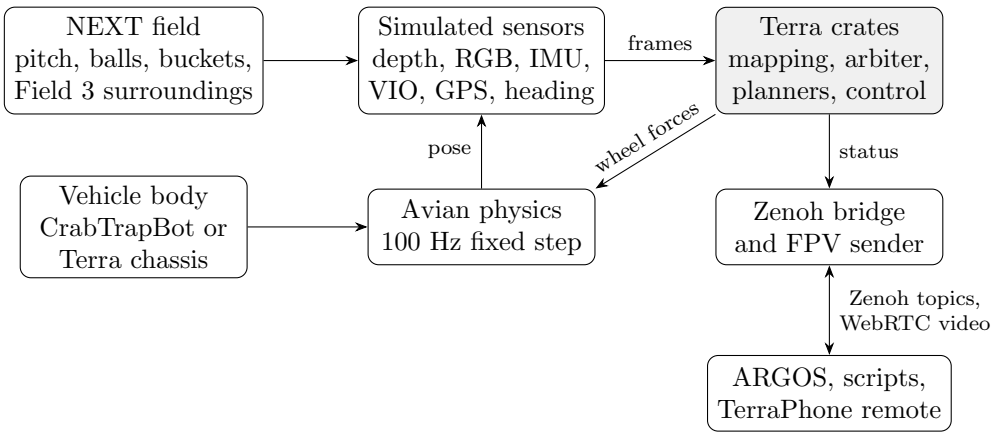


Figure 4.1: Zorvane's structure. The shaded box is Terra's own code, linked unchanged. Everything a client sees passes through the bridge, on the same topics a real rover uses.

4.3.1 Vehicle bodies

A vehicle is a plug-in: a body declares its identifier, its visual model, its chassis box and mass, and its locomotion. Two bodies are registered: CrabTrapBot, the default NEXT rover, and the Terra development chassis (Chapter 3). Both are differential drive; the simulator turns a differential-drive body into a wheel controller and a physics configuration, and refuses at startup

to drive a body whose locomotion it does not support. That refusal is the extension point for other vehicle types, such as a drone, without forking the world or the topic layout.

4.3.2 Physics and drive

Each rover is a single dynamic rigid body, a box the size of the chassis, with yaw free and roll and pitch locked. Wheel commands become forces on that body, with linear, lateral, and yaw drag tuned so that the rover accelerates, coasts, and resists sideslip plausibly. Physics and control run on a 100 Hz fixed step. Wheel odometry is ideal. There are no wheel joints, no tyre model, and no suspension; skid-steer scrub, for example, is represented only through drag.

4.3.3 Sensors

Each rover carries a front-mounted depth sensor and a colocated RGB camera, both rendered on the GPU and read back asynchronously. The default depth sensor is 256 × 192 pixels with a 60° vertical field of view and a range of 0.05 to 30 m, and reports axial depth in metres with NaN for no return. It is an ideal sensor: it has no noise and no housing, and it sees opaque and alpha-masked geometry, including meshes with no collider. Depth frames carry the camera and body pose at exposure time, so the rover's occupancy map is built from exposure-aligned frames exactly as on the phone.

The velocity controller receives phone-like feedback, an IMU sample every physics step and a visual–inertial pose at 20 Hz, taken from the simulated body. Because the Field 3 placement gives the pitch a geographic origin and heading, the simulator also produces GPS fixes and a true heading from the rover's pose. These are exact: they are sampled from ground truth and reported with an accuracy of 1 m and 1°. Section 4.6 returns to what that means for outdoor results.

4.3.4 The bridge

The bridge is the simulator's only external interface. It publishes and subscribes on the same Zenoh topics as TerraPhone, under a configurable prefix, so a client can be pointed at Zorvane, at a real rover, or at the cloud router without changes. It applies the same command rules the phone applies. Every non-neutral command must carry the current run identifier and authority revision. A teleoperation command expires after 500 ms. A stop dominates every other request received in the same control tick, including a reset, and a full command queue drops the oldest non-stop request first. Messages larger than 2048 bytes are ignored.

Telemetry rates match the phone's defaults: pose and camera frames at up to 10 Hz, status topics at about 5 Hz, and occupancy as full snapshots when the map changes, or at least every two seconds, with lightweight heartbeats in between. First-person video is H.264 over WebRTC, using the same signalling as TerraPhone, and the camera renders only while a call is live. A rover can be armed and disarmed, and the simulated actuator relay reports readiness like the real Pi bridge.

4.3.5 Game logic

The NEXT rules in Section 2.2 are implemented as simulator systems: the intake that captures balls entering CrabTrapBot's opening, the store and its capacity, the waiting state at the goal line, operator-triggered loading into the dumper, the dump readiness check, ball physics into the bucket, and the score. The two game actions, load and dump, are also Zenoh topics, and `next/status` reports each rover's stage (searching, carrying balls, full and returning, or waiting for the operator), so the game loop can be driven and observed from ARGOS or a script.

4.4 Running the simulator

Zorvane runs as a desktop application with a start menu, in which the user picks the number of rovers, followed by a narrated cutscene of the rules. It also runs headless, with no window, for automated checks. Release builds for macOS, Windows, and Linux are produced by continuous integration from a version tag, so an operator or study participant can run the simulator without a Rust toolchain; the first tag is v0.1.0, from October 24, 2026. Environment variables select the rover count, the topic prefix and listener, the bucket seed, the lighting, extra balls, the camera view, and whether video is on.

For a study session the simulator can connect to the same authenticated cloud router as the real fleet, under its own prefix, so that the operator station is configured once. [**TODO: Describe the exact session setup for simulated trials: local listener or cloud router, rover count, seed list.**]

4.4.1 An AI model as a player

Zorvane can also be played by a language model instead of a person (Zorvane PR #26, open and unmerged at the time of writing). A small server exposes one rover through the Model Context Protocol (MCP), so any MCP client can call it as a set of tools: read the match state and score, look through the forward camera as an image, read the occupancy map as text, choose an autonomy level from L0 to L4, send a waypoint goal at L2 or L3, drive a teleoperation burst of at most 5 s at up to 2 m/s, stop, and press the NEXT load and dump buttons. Bucket positions are not exposed, so the model has to find buckets by looking. The server uses the same Zenoh topics and the same run and authority binding as ARGOS, so the model's commands pass the rover's checks exactly as an operator's do. In a headless test run, a model client read the match state, took a camera image, drove, changed level, and reached an L2 goal; L3 goals and the text map were not verified, because the software-rendered run published no occupancy map, and loading and dumping were not exercised.

This differs from L5 (Section 5.8.8). In L5 the model runs on the phone and only proposes waypoints to the arbiter; here the model runs off-board and plays the role of the operator, including the choice of autonomy level. That makes it a way to study level choice itself: a model, or a learned policy, can choose the level for each phase of a match, and its choices can be compared with an operator's. [**TODO: Merge the MCP server, verify L3 goals and the map on a GPU machine, and decide whether model-played matches become part of the evaluation.**]

4.5 Logging and repeatability

Every simulated run writes the same JSONL run log as the phone, through Terra's run recorder: the run identity and configuration, each control tick's output, authority events, mission progress, observed free area, contacts, and near misses. The simulator also sends diagnostic events to the observability collector described in Chapter 2.

Repeatability comes from three things: the pinned Terra revision, a fixed seed for the bucket layout (and for any extra balls), and a fixed rover spawn on the goal lines. Rendering and physics run on the host's GPU and CPU, so two runs with the same seed are not guaranteed to be bit-identical, but they start from the same state and follow the same rules. [**TODO: Measure run-to-run variation for a fixed scripted mission, to state how repeatable a simulated trial is.**]

The test suite runs without a graphics device; tests that need a real GPU, such as checking a rendered wall distance in metres, are marked to run only on request.

4.6 Fidelity and the reality gap

Table 4.1 lists what Zorvane models and what it simplifies or omits, and what each gap means for the thesis.

Table 4.1: Simulator fidelity, from the simulator's own code and documentation.

Aspect	In Zorvane	Consequence
Interface	Same topics, payloads, command rules, rates, and video signalling as TerraPhone	Operator-facing behaviour transfers directly; this is what the simulator is built to get right
Autonomy code	Terra crates linked unchanged at a pinned revision	Planner and arbiter logic is the same; its inputs are not
Field	100 × 64 m pitch at Field 3's geographic placement; OpenStreetMap surroundings; flat	Distances, layout, and GPS frame match; terrain does not
Depth	Ideal GPU depth, 256 × 192, no noise	Obstacle avoidance looks better than on a phone with noisy or missing depth
Pose	IMU and VIO from ground truth; ideal odometry	No VIO drift or tracking loss
GPS and heading	Exact, reported at 1 m and 1°	The GPS accuracy gate and the heading problem (Section 2.3.5) never trigger; outdoor results must come from Field 3
Physics	One rigid box, roll and pitch locked, drag-based skid	No tyre slip, tipping, or turf effects; turning limits must be calibrated on the real rover
Mass and motor limit	20 kg and a 2.29 m/s cap, both unsourced placeholders	Speeds and accelerations are indicative only
Ball handling	Idealized intake and dump chute	Scoring in simulation is an upper bound on the real mechanism
Rules	Scoring for ground buckets only; launching, tagging, and penalties not enforced	Simulated matches are practice, not full matches
Network	Local or router link; no injected loss	Communication loss must be added deliberately for a study

The pattern is consistent: Zorvane is faithful where the operator meets the system, and idealized where the robot meets the world. That suits the thesis's question, which is about how an operator supervises robots at different levels of autonomy, more than about how well any one robot perceives. It also sets a clear rule for reporting. Interface-level measures from the simulator, such as how often an operator switches level, how quickly they respond, and how many interventions a mission takes, are meaningful as practice and pilot data. Robot-level measures, such as obstacle clearance, navigation success, and GPS arrival accuracy, are reported only from the real rover. [**TODO: Add sensor noise, VIO drift, GPS error, and link loss as configurable options, and report which were enabled for any simulated result.**]

4.7 History

The simulator began inside the Terra repository as a Bevy world for developing the rover's control and autonomy, and was moved into its own repository, Zorvane, with the shared robotics crates left in Terra. Over October 2026 it held several worlds: a practice town, real elevation tiles around George Mason University, photorealistic 3D-tile scenery, a seeded disaster-search mission with broken walls, rubble, and three hidden simulated survivors, and a browser build. On October 24, 2026, Zorvane was reduced to the single NEXT field; the other worlds, the survivor mission and its detector, and the browser build were removed. Terra's documentation still describes the disaster-search mission, and its mission bookkeeping remains in the shared code, which ARGOS still reads. [**TODO: Decide whether the disaster-search mission is needed for the study; if so, restore it from history at a pinned revision and describe it here.**]

4.8 Summary

Zorvane is a Rust simulator of the NEXT field that links Terra's autonomy unchanged and speaks exactly the protocol a real rover speaks. It supports algorithm and interface development, fleets larger than the team can build, and practice on the competition field. Its sensors, pose, GPS, and physics are idealized, so it is used for operator-facing measures and pilot work, and every robot-level claim in this thesis is checked on the real rover. The next chapter describes the software architecture and the autonomy levels that run in both places.

Chapter 5

System Architecture and Autonomy

This chapter describes the software that runs the rover and the mathematics behind each autonomy level. Section 5.1 outlines the architecture. The rest of the chapter gives the models and control laws in the order a command passes through them: state estimation and mapping (Sections 5.3–5.4), the vehicle model and velocity controller (Section 5.5), the turning policy (Section 5.6), the local planner (Section 5.7), and then the intent each level produces (Section 5.8). Section 5.9 covers outdoor GPS guidance and heading fusion, and Section 5.10 a prototype that fuses two rovers' maps.

Every equation in this chapter is taken from the Terra source code, and every default value is the one in the code as of October 2026. They are software starting points, chosen in simulation; Chapter 3 lists the physical limits that still have to be measured.

5.1 Architecture

The rover software has two layers on the phone. TerraPhone is a Swift application. It owns everything that needs an Apple framework: the ARKit session and depth frames, Core Motion, GPS and compass, the Bluetooth link to the motor bridge, WebRTC first-person video, and, for L5, the on-device language model (Section 5.8.8). Below it is the Terra core, a set of Rust crates compiled for iOS and called through UniFFI bindings. The same crates are linked by the Zorvane simulator (Chapter 4), so the autonomy that runs on the phone is the autonomy that runs in simulation. An Android client for the same Bluetooth actuator protocol also exists, but the phone used in this thesis is an iPhone.

5.1.1 Where the control loop belongs

The project follows one rule for splitting work between the two languages: Swift gathers sensor data, talks to the hardware and the network services that only Apple's frameworks provide, and builds the occupancy input from depth; the control loop, safety gating, mode and tick policy, and actuator routing belong in Rust. Rust code is shared with the simulator and covered by its tests, so a rule that lives there is exercised in both places; a rule in Swift runs only on a phone.

In October 2026 the code is part of the way there. The arbiter, planners, velocity controller and level logic are in Rust, but TerraPhone still ticks the loop and still holds the motor gates and

the mapping from wheel effort to actuator outputs. The first migration step (Terra PR #63) makes Rust route each tick's efforts through the rover's drive profile as well and return the resulting actuator frame; TerraPhone still sends its own frame and logs any tick where the two differ. This *shadow mode* lets the Rust routing be checked against the Swift routing on real drives before it takes over. **[TODO: Report how many shadow-mode mismatches were logged on real drives, then switch the actuator output to the Rust frame and describe the final split.]**

Two fixes from the same period show why the loop's timing is delicate on a phone. ARKit delivered frames on the same queue as the 100 Hz control tick, depth mapping and point-cloud publishing, so frames that arrived during a step waited holding their `ARFrame`, and ARKit warned that eleven frames were retained (fixed in Terra PR #65). A semaphore wait between the high-priority frame thread and a utility-priority capture queue was a priority inversion (fixed in Terra PR #61). Neither changes the mathematics below, but both affect how fresh the inputs to it are.

An audit of TerraPhone and its Rust bindings on October 27, 2026 measured how far the split still has to go. The Rust `step` already returns the routed actuator frame, but iOS only compares it with its own and still encodes and sends the Swift frame; the Android client already sends the Rust one. The e-stop latch, the mount-confirmation gate, the pause on lost visual tracking and the tick dividers exist only in Swift, and iOS and Android use different conventions for the phone-to-body mount transform. The audit also found bugs in the Swift layer, the most serious being that stopping the controller cleared the e-stop latch, so a stop followed by a restart silently re-armed a rover that had been e-stopped. Those bugs and the dead code around them were fixed in Terra PR #83: the latch now survives a stop and is cleared only by an explicit reset. The remaining order of work is to send the Rust actuator frame on iOS, give the Rust tick a typed result in place of a status string, and move the gates and tick dividers into Rust. **[TODO: Report the migration once iOS sends the Rust actuator frame.]**

5.1.2 Loop timing on the phone

Every stage of the pipeline in this chapter holds its output when an input is stale: the estimator when the IMU is older than 0.1 s, the controller when its time step exceeds 0.1 s, the turning limiter when its step exceeds 0.1 s. These rules make the rover stop rather than act on old data, but they also turn any delay on the control queue into a hold, and a hold clears the controller's integrators. Three faults found during the motion-capture sessions of October 26–27 (Section 5.3.1) were of this kind, and none was a fault in the control law itself.

Mesh walk on the control queue. The floor veto of Section 5.4.3 first walked the ARKit mesh on the control queue once a second. Each walk stalled pose and IMU delivery for about 150 ms, so the estimator reported itself not ready, the controller held, and a commanded 0.4 in effort fell to about 0.19 after each reset. The walk now runs on a utility queue and is built outside the map lock (Terra PR #77), and walks in flight are dropped after a map or ground reset (Terra PR #78).

Status parsing at L3. In L3 the autonomy status carries the planner trace, about 30 KB of

JSON, and TerraPhone parsed it three to four times on every 10 ms tick. The control queue fell behind, the IMU sample went stale, and the rover held with `sensor_unhealthy`; one L3 run logged 191 such holds. Rust now also returns a short autonomy brief of under 1 KB that the hot Swift paths read instead (Terra PR #82). Before the fix, ARGOS could only show that the rover was holding; now Terra records the cause of each sensor hold and keeps it for 10 s, and ARGOS shows it on a *Sensor hold* row (Terra PR #82, ARGOS PR #89). The cause read `stale_imu`, which is what located the fault.

Late ticks in straight runs. In the fourth motion-capture session, five straight runs ended about one second after they started, when one tick arrived more than 0.1 s late and the turning limiter held. The teleoperation client then treated the hold as a release of the drive keys. This is probably the same status-parsing load, since that phone build did not yet have Terra PR #82, but it has not been confirmed.

The holds explain why L3 failed to reach a goal that L2 reached from the same start. L2 bypasses the planner, while the L3 planner's velocity window was centred on the measured speed (Section 5.7): after each hold the measured speed of a wheel inside the motor deadband was zero, the planner offered at most 0.1 m/s, and the wheels never left the deadband. [**TODO: Measure the control-tick latency distribution on the phone at L0 and L3 after Terra PR #82, and report how often a tick exceeds the 0.1 s limits.**]

5.1.3 Fleet communication

Every rover publishes and subscribes under its own numeric identifier below one shared Zenoh prefix (Chapter 2). Light telemetry, such as pose, localization, goal and autonomy status, hardware state, and outdoor navigation, is received for every rover. Heavy topics, the occupancy grid, the point cloud and depth frames, are subscribed only for the rover the operator has selected (ARGOS PR #69), so the router does not forward every rover's map to the operator station. Requests that are not motion, such as a point-cloud capture request, go to their own inbox outside the motion-authority path, and bulk replies are sent at background priority and dropped rather than queued when the link is congested. [**TODO: Describe run identity and authority revisions, the Pi Bluetooth protocol and logging in an appendix, and list the topic contract.**]

5.1.4 Motion authority

For the rest of this chapter one property of the architecture matters: all motion authority sits in one place, the *arbiter* in the Terra core on the phone. Operator commands, waypoint goals, and autonomous goals are all *intents*. The arbiter turns the intent of the active level into a velocity command, checks it, and hands it to the velocity controller. The control loop ticks every 10 ms; wheel commands are sent to the motor bridge every fifth tick, at 20 Hz; the local planner works on a 0.1 s step.

5.2 Notation and frames

The rover pose in the local map frame is $\mathbf{p} = (x, y, \theta)$, with x, y in metres and yaw θ in radians, measured counter-clockwise from the $+x$ axis. The body twist is (v, ω): forward speed in m/s and yaw rate in rad/s, positive to the left. A hat marks an estimate ($\hat{v}$), a star a target (v^*), and a subscript i the intent produced by an autonomy level (v_i, ω_i). $\mathrm{clip}(a, \pm b)$ limits a to $[-b, b]$, and $\mathrm{wrap}(\cdot)$ maps an angle to $[-\pi, \pi)$. Geographic headings are in degrees, clockwise from true north.

5.3 State estimation

The velocity controller needs the rover's forward speed and yaw rate. The phone has no wheel encoders in the loop, so Terra estimates them from ARKit's visual–inertial odometry (VIO) and the Core Motion IMU. An independent benchmark found ARKit the most stable and accurate of four commercial VIO systems [19], so its pose is a reasonable stand-in for wheel odometry. The latest VIO sample, at time t_0, gives an orientation quaternion $\mathbf{q}_0$ and a world-frame velocity $\mathbf{v}_0$. Because VIO arrives late and at a lower rate than the IMU, the estimator replays the retained IMU samples $(\mathbf{a}_k, \boldsymbol{\omega}_k)$ from t_0 to the current time. For each interval δ_k,

$$\mathbf{q}_{k+\frac{1}{2}} = \mathbf{q}_k \otimes \exp\left(\tfrac{1}{2}\boldsymbol{\omega}_k \delta_k\right), \tag{5.1}$$

$$\mathbf{v}_{k+1} = \mathbf{v}_k + R(\mathbf{q}_{k+\frac{1}{2}})\, \mathbf{a}_k\, \delta_k, \tag{5.2}$$

$$\mathbf{q}_{k+1} = \mathbf{q}_k \otimes \exp(\boldsymbol{\omega}_k \delta_k), \tag{5.3}$$

where $\exp(\cdot)$ maps a rotation vector to a unit quaternion and $R(\mathbf{q})$ is the corresponding rotation. The estimate is the body-frame forward component and the latest gyro yaw rate:

$$\hat{v} = \left[R(\mathbf{q}_N)^\top \mathbf{v}_N\right]_x, \qquad \hat{\omega} = \omega_{z,N}. \tag{5.4}$$

The estimate is withheld, and the controller stops, if VIO reports lost tracking, if the IMU is older than 0.1 s, or if VIO is older than 0.35 s. On the phone, the VIO velocity $\mathbf{v}_0$ is itself a finite difference of the ARKit position of the rover's reference point, low-pass filtered with a 30 ms time constant.

The IMU vectors must be rotated from the phone's axes into the rover body through the mount calibration (Chapter 3). Until Terra PR #74 the iOS IMU path assumed a phone lying flat, so on an upright mount the gyro axis read as yaw was the phone's roll axis, and the estimated yaw rate stayed near zero through arcs at 0.5 to 1.1 rad/s. The fix maps Core Motion through the saved mount transform. That transform is only as good as the saved mount: with the landscape phone on CrabTrapBot, a calibration with roll 0 leaves the yaw-rate axis and body up 90° off. The ARKit heading and forward speed do not depend on this mapping and were unaffected.

5.3.1 Validation against motion capture

Neither chassis has wheel encoders, so the VIO speed is the only speed the controller sees. To check it, CrabTrapBot was driven in an OptiTrack motion-capture volume of about 12×8 m on October 26 and 27, 2026. OptiTrack (Motive 3, NatNet 4) tracked a rigid body on the chassis at 120 Hz; a 3 m tape measure read 0.01 m different from OptiTrack, so it served as ground truth. A bridge script republished the rigid-body pose on the fleet router, and a single program on the laptop recorded both poses and drove the rover in straight lines at a set speed and distance through ARGOS-style teleoperation. The phone's observability journal was copied off the phone after each session. For every run, the steady part of the phone's forward speed, after the speed-up and before the stop, was compared with OptiTrack's.

Table 5.1 gives the ratio of phone to OptiTrack speed for the 17 usable straight runs at 0.2 to 0.4 m/s.

Table 5.1: Steady forward speed, phone VIO divided by OptiTrack, for straight runs on CrabTrapBot, October 26–27, 2026.

Session	Runs	Ratio per run
1 (Oct 26)	2	0.98, 0.81
2 (Oct 26)	4	0.81, 0.89, 0.93, 0.88
3 (Oct 26)	1	1.01
4 (Oct 27)	10	0.83, 0.60, 0.93, 0.95, 0.92, 0.86, 0.90, 1.01, 0.92, 0.97
All	17	mean 0.894, s.d. 0.098, median 0.92
Without the 0.60 run	16	mean 0.912, s.d. 0.065

The phone under-read the speed in 15 of the 17 runs. The same under-read appeared when the phone was carried by hand along a line, 4.06 m on the phone against 4.42 m on OptiTrack (0.92), so it comes from ARKit and not from the wheels or motors. The other properties the controller needs were good: at rest the speed bias was 0.0009 m/s with a standard deviation of 0.003 m/s; while driving the noise was 0.007 m/s in the control band and 0.012 m/s above it; and ARKit's heading agreed with OptiTrack to within 0.1° over a 475° arc. The latency relative to the gyro was roughly 50 to 70 ms, a rough figure because the journal logged at only 8 Hz. Two events fell outside this behaviour: the 0.60 run and a 2.75 m jump in ARKit position on one arc, both consistent with VIO losing and re-finding its scale or anchor.

For the controller of Section 5.5, a scale error of about 9% shifts the loop gain by the same small amount and leaves it stable, but the loop drives the *measured* speed to the target, so the rover really travels about 9% faster and farther than it believes. Over a 1 s planner rollout at 0.3 m/s that is 3 to 4 cm, less than one grid cell. The occupancy map is built from LiDAR depth in the same ARKit frame, so the rover stays consistent with its own map; absolute distances, GPS goals and the fusion of two rovers' maps are where the error shows. The leading explanation is the room itself: OptiTrack's infrared strobes may interfere with the phone's LiDAR, and the floor was plain. [**TODO: Repeat the straight runs in a textured hallway and outdoors with a tape measure; if the ratio stays near 0.91, add a fixed scale correction to the velocity estimate.**] [**TODO: Measure arcs and turns after setting the landscape mount roll, log at 100 Hz to tighten the latency estimate, and sweep 0.1 to 0.5 m/s.**]

[TODO: Add a guard that rejects VIO position steps larger than the rover could physically move, and size it from longer sessions.]

5.4 Occupancy mapping

The local map is a rolling grid of 200×200 cells at resolution $r = 0.1$ m. Each cell holds a log-odds value ℓ. For each depth frame, every sampled pixel (u, v) with depth z is back-projected through the camera intrinsics,

$$\mathbf{r}_c = \left(\frac{(u - c_x)z}{f_x},\ \frac{(v - c_y)z}{f_y},\ z \right), \tag{5.5}$$

rotated into the world frame, and classified as an obstacle hit if its endpoint lies between 0.15 and 2.0 m above the ground plane, within 15 m of the camera. Each pixel gets one of six labels: free, obstacle, overhead, out of range, outside the band, or no return, plus the floor-veto label of Section 5.4.3; the grid update below uses the same classifier, so the labels and the map evidence cannot disagree. The cells along the ray, found by Bresenham traversal, are marked free, and the endpoint cell is marked occupied for a hit. Each cell is updated at most once per frame, with obstacle evidence taking precedence:

$$\ell \leftarrow \mathrm{clip}\left(\ell + \begin{cases} +0.85 & \text{occupied} \\ -0.40 & \text{free} \end{cases},\ \pm 4 \right). \tag{5.6}$$

The map published to the planner stores $P = 100\,\sigma(\ell) = 100/(1 + e^{-\ell})$ for observed cells and -1 for unobserved ones. A cell is *occupied* if $P \geq 65$, that is if $\ell \geq \ln(65/35) \approx 0.62$, *free* if $0 \leq P < 65$, and *unknown* otherwise. The asymmetric increments mean that one hit outweighs two misses, so a thin obstacle seen intermittently stays in the map.

5.4.1 The ground plane

Every height in the classifier is measured from a ground plane, so an error in the plane becomes an error in the map. The plane is level, with normal along world up, and lies a vertical distance h_c below the camera:

$$z_{\text{ground}} = z_{\text{cam}} - h_c, \tag{5.7}$$

so it follows the terrain's elevation as the rover moves, but never its tilt. The body's up axis is used only to reject a tipped-over rover. An earlier version used the body's up axis as the plane normal. A labelled point capture on a flat floor showed the body axis about 16° off gravity because of mount pitch and roll, so the plane cut through the floor: on one side, floor beyond about 2 m was labelled obstacle, and on the other, walls up to 2.5 m high read as free (fixed in Terra PR #66).

The camera height h_c comes from the mount calibration (Chapter 3) when one with a plausible height (0.1 to 3 m) is saved, as a robot model would use a fixed camera transform (Terra PR #68). Without a calibration, h_c is taken once from the first ARKit horizontal plane of at least 0.4 m^2 whose height stays within 3 cm for 1 s. That fallback is fragile: if the app

is started with the phone in hand and the phone is mounted afterwards, h_c is the hand height and the plane sinks below the real floor.

Before a floor is found, the map uses a *horizon band*: returns within $\pm 10^\circ$ of the horizon count as obstacles out to the range at which the band meets the floor, $h_c / \tan 10^\circ$, about 1.3 m for a 0.23 m camera. The band has no margin, so a camera height set a little too high puts that limit beyond the true floor intercept: in one capture in the motion-capture room every one of the 4,094 obstacle points was floor, in a ring 1.1 to 1.3 m around the rover. The band also persists if the ground plane rejects the body's up axis, which it does when the body appears tilted by more than 60°. On CrabTrapBot that happened because the phone is mounted in landscape and the mount calibration still had roll 0, so the body appeared to lie on its side. The map status now reports the tilt, and band frames now use the same height offset as the mesh floor, which had been drawn 0.23 m too high (Terra PR #82). [**TODO: Repeat the flat-floor point capture with the level, mount-calibrated plane on CrabTrapBot and report the fraction of floor points labelled obstacle before and after.**]

5.4.2 Inspecting the classifier

The ground-plane fault was found with a diagnostic built for that purpose. On request from ARGOS, the rover sends every depth pixel, projected to world coordinates and labelled by the rules above, as binary frames (Terra PR #64, ARGOS PR #72). A *Capture 3D* records 10 s at 1 Hz, about 640 KB per frame, and ARGOS saves it as a PLY file coloured by label. A *Live 3D* stream sends decimated frames at up to 5 Hz for as long as ARGOS renews a lease, and ARGOS draws the newest frame on the map under the rover (Terra PR #67, ARGOS PR #83); ARGOS uses a 6 s lease renewed every 2 s, 2 Hz, and every third pixel in each direction, about 70 KB per frame. Decimation scales the camera intrinsics so that every ray kept is unchanged. The always-on thinned point cloud that the phone used to stream is now sent only while a capture runs. For work without a rover or a laptop, TerraPhone also has a *3D classification* debug page that shows the live labelled point cloud and the mesh floor cells on the phone itself, held in the hand (Terra PRs #80 and #81).

5.4.3 Floor veto from the ARKit mesh

A level ground plane removes the tilt error but not every false obstacle: depth noise, a slightly wrong camera height or a gentle slope can still lift a floor return above the 0.15 m band floor. ARKit can classify its reconstructed scene mesh into surfaces such as floor, wall, window and door, and this gives a second, independent estimate of where the floor is. TerraPhone turns on the classified mesh and sends the centres of floor faces to the Rust map once a second. The map bins them into 0.1 m cells, keeps the median height per cell, and drops cells more than 8 cm from the median of the whole surface. In the classifier, a return that would be an obstacle but lies less than 0.15 m above a trusted mesh floor cell is relabelled *floor veto* and clears its cell like a free return; a return higher than that above the same cell is still an obstacle, so an object standing on the floor is not erased. The mesh floor also seeds the ground-plane height when no floor plane anchor has been found. Captures carry the same surface, so a captured frame shows exactly what the map did, and ARGOS colours the new label (Terra PR #76, ARGOS

PR #87, both merged October 26, 2026). Two design questions remain open: whether a veto should count as free evidence or as no evidence, and how well ARKit's floor classification holds on turf. [**TODO: Repeat the flat-floor capture with the floor veto on and report the fraction of floor returns labelled obstacle with and without it.**]

5.4.4 Glass

The phone's LiDAR sees through clear glass, so the map carves glass walls and doors as free space, and L3 would route through them. A change merged on October 27, 2026 uses the same classified mesh: faces ARKit labels wall, window or door are sent to the map, and cells with at least three such faces are marked as a *barrier* with a height span. A depth ray that crosses a barrier cell stops there; the cell is marked occupied and the return gets a new barrier label (Terra PR #84, ARGOS PR #90). This only works where ARKit has put mesh faces on the glass, which a clear pane seen at a shallow angle may not get. [**TODO: Test the barrier on real glass doors and windows; if it misses them, compare remembering bumps and adding an ultrasonic sensor.**]

5.5 Vehicle model and velocity control

5.5.1 Kinematics

Both chassis are differential drive. Treated as a unicycle,

$$\dot{x} = v\cos\theta, \qquad \dot{y} = v\sin\theta, \qquad \dot{\theta} = \omega, \tag{5.8}$$

with the wheel speeds related to the body twist by

$$v = \frac{v_R + v_L}{2}, \qquad \omega = \frac{v_R - v_L}{b_{\text{eff}}}, \tag{5.9}$$

where b_{eff} is the effective track width. On a skid-steer chassis b_{eff} is larger than the geometric track because the wheels scrub sideways in a turn. The controller below never uses b_{eff} explicitly: it works in normalized wheel effort, and the track width, motor gain, and scrub are absorbed into its feedforward and feedback gains. Equation (5.8) is the model the planner uses to predict trajectories.

5.5.2 Velocity controller

The velocity controller turns a target twist (v^*, ω^*) into normalized left and right wheel efforts $u_L, u_R \in [-1, 1]$. Targets are first limited to $|v^*| \le 2$ m/s and $|\omega^*| \le 2$ rad/s. With errors $e_v = v^* - \hat{v}$ and $e_\omega = \omega^* - \hat{\omega}$, the linear and yaw efforts are feedforward plus proportional–integral terms,

$$u_v = k_v^{\text{ff}} v^* + k_v^p e_v + I_v, \tag{5.10}$$

$$u_\omega = k_\omega^{\text{ff}} \omega^* + k_\omega^p e_\omega + I_\omega. \tag{5.11}$$

They are mixed into wheel efforts and scaled down together, which preserves the ratio between turning and driving when the command saturates:

$$s = \max\left(1,\ \frac{\max(|u_v - u_\omega|,\ |u_v + u_\omega|)}{u_{\max}}\right), \qquad u_L = \frac{u_v - u_\omega}{s}, \quad u_R = \frac{u_v + u_\omega}{s}. \tag{5.12}$$

Unless turning in place is allowed, the wheels may not counter-rotate: if $u_L u_R < 0$, the wheel opposing the mean direction $\frac{1}{2}(u_L + u_R)$ is set to zero. The integrators use back-calculation anti-windup against the effort actually applied, $\bar{u}_v = \frac{1}{2}(u_L + u_R)$ and $\bar{u}_\omega = \frac{1}{2}(u_R - u_L)$:

$$I_v \leftarrow \mathrm{clip}\big(I_v + \left[k_v^i e_v + k_{\mathrm{aw}}(\bar{u}_v - u_v)\right]\Delta t,\ \pm u_{\max}\big), \tag{5.13}$$

$$I_\omega \leftarrow \mathrm{clip}\big(I_\omega + \left[k_\omega^i e_\omega + k_{\mathrm{aw}}(\bar{u}_\omega - u_\omega)\right]\Delta t,\ \pm u_{\max}\big). \tag{5.14}$$

When the target is zero and the rover is nearly still ($|\hat{v}|, |\hat{\omega}| < 0.015$), the outputs are forced to zero and the integrators reset, so the motors do not hum at rest. If in-place turning is allowed and the target is a pure rotation, the controller bypasses the loop and applies a fixed spin effort, $u_R = -u_L = u_s \operatorname{sgn}\omega^*$. Small DC motors do not turn below some effort d, so a rover profile may set a wheel deadband (Terra PR #71). Each moving wheel's effort u is then lifted past it before it is sent,

$$u' = \operatorname{sgn}(u)\left(d + (u_{\max} - d)\,\frac{|u|}{u_{\max}}\right) \quad \text{for } |u| \geq 0.01, \tag{5.15}$$

while efforts under 0.01 stay zero and the integrators work on the uncompensated effort. The default is $d = 0$; Chapter 3 gives the per-rover values.

The controller stops (zero effort, integrators cleared) when the target is 0.5 s old or older, when the time step is non-positive or longer than 0.1 s, or when the estimator is not healthy. Table 5.2 lists the defaults. With $k_v^{\mathrm{ff}} = 1/3$, full effort corresponds to a feedforward speed of 3 m/s.

Table 5.2: Velocity controller defaults (`ControllerConfig`). **[TODO: tune on CrabTrapBot and report the tuned values]**

Symbol	Meaning	Default
$k_v^p,\ k_v^i$	Linear proportional and integral gains	0.3, 0.6
$k_\omega^p,\ k_\omega^i$	Yaw proportional and integral gains	0.1, 0.3
$k_v^{\mathrm{ff}},\ k_\omega^{\mathrm{ff}}$	Linear and yaw feedforward	1/3, 0.2
k_{aw}	Anti-windup gain	4.0
$u_{\max}$	Maximum effort	1.0
u_s	In-place spin effort	0.35
d	Wheel deadband	0
–	Target timeout; maximum step	0.5 s; 0.1 s

5.6 Turning policy

Skid-steer turns scrub the tyres and disturb the phone's visual tracking, so every *managed* command, meaning ARGOS teleoperation and every autonomous level, passes through a turning

limiter before the controller. Local joystick driving on the phone does not. Given a requested ω^* at forward speed v, the limiter first caps the yaw rate by a minimum turning radius $R_{\min}$,

$$\omega_{\text{cap}} = \begin{cases} \omega_{\max} & \text{in-place turns allowed,} \\ \min(\omega_{\max},\ |v|/R_{\min}) & \text{otherwise,} \end{cases} \tag{5.16}$$

so that the rover cannot turn on the spot by default. It then slews from the previous command ω_{prev} toward the capped request $\omega_d = \text{clip}(\omega^*, \pm\omega_{\text{cap}})$, passing through zero on a reversal:

$$g = \begin{cases} 0 & \omega_{\text{prev}}\,\omega_d < 0, \\ \omega_d & \text{otherwise,} \end{cases} \qquad \alpha = \begin{cases} \alpha_{\text{dec}} & \omega_{\text{prev}}\,\omega_d < 0 \text{ or } |g| < |\omega_{\text{prev}}|, \\ \alpha_{\text{acc}} & \text{otherwise,} \end{cases} \tag{5.17}$$

$$\omega = \text{clip}(\omega_{\text{prev}} + \text{clip}(g - \omega_{\text{prev}},\ \pm\alpha\Delta t),\ \pm\omega_{\text{cap}})\,. \tag{5.18}$$

The defaults are $\omega_{\max} = 0.5$ rad/s, $\alpha_{\text{acc}} = 0.5$ rad/s^2, $\alpha_{\text{dec}} = 0.8$ rad/s^2, $R_{\min} = 0.5$ m, and in-place turning off; reverse speed is limited to 0.15 m/s. When an autonomous level needs to face a new direction from rest, the arbiter plans a multi-point manoeuvre of forward and reverse arcs that respects $R_{\min}$, rather than spinning.

5.7 Local planner

The local planner is the safety layer for every level that checks obstacles. It is a deterministic velocity-space sampler in the family of the dynamic window approach [14]: it searches the velocities reachable in one step for the one closest to the intent whose predicted path stays clear.

5.7.1 Dynamic window

The window grows from a centre twist $(\bar{v}, \bar{\omega})$. For each axis the centre is the last command the arbiter allowed, $(v_{\text{prev}}, \omega_{\text{prev}})$, zero after a hold, unless the rover is already measured going faster in the same direction:

$$\bar{v} = \begin{cases} \hat{v} & v_{\text{prev}}\,\hat{v} \geq 0 \text{ and } |\hat{v}| > |v_{\text{prev}}|, \\ v_{\text{prev}} & \text{otherwise,} \end{cases} \tag{5.19}$$

and likewise for $\bar{\omega}$. With planning step $\Delta t = 0.1$ s, the reachable window is

$$V = [\bar{v} - a\Delta t,\ \bar{v} + a\Delta t] \cap [-v_{\text{rev}},\ v_{\max}], \tag{5.20}$$

$$\Omega = [\bar{\omega} - \alpha\Delta t,\ \bar{\omega} + \alpha\Delta t] \cap [-\omega_{\max},\ \omega_{\max}]. \tag{5.21}$$

Until Terra PR #82 the window was centred on the measured twist alone. A wheel inside the motor deadband measures zero, so after a hold the planner offered at most $a\Delta t = 0.1$ m/s, which kept the wheels in the deadband; this is how L3 stalled where L2 did not (Section 5.1.2). The planner samples an 11×11 grid of $(v, \omega) \in V \times \Omega$, evenly spaced including the endpoints. For managed commands each sample is first passed through the turning limiter of Section 5.6,

so the planner only considers commands that will actually be executed.

5.7.2 Trajectory rollout with braking

Each candidate is rolled forward under Equation (5.8) for the prediction horizon H, followed by a braking tail long enough to stop at deceleration a_b:

$$T = H + \frac{|v|}{a_b}, \qquad \lambda(t) = \begin{cases} 1 & t \le H, \\ \max\left(0,\ 1 - \dfrac{(t-H)\,a_b}{|v|}\right) & t > H. \end{cases} \tag{5.22}$$

Both velocities are scaled by $\lambda(t)$, which preserves the path's curvature while braking. With step $\delta = \min(r/(2\max(|v|, 0.1)),\ 0.1)$, so that the rover advances at most half a cell per step,

$$\theta_{n+1} = \theta_n + \lambda\omega\delta, \quad x_{n+1} = x_n + \lambda v \cos\theta_{n+1}\,\delta, \quad y_{n+1} = y_n + \lambda v \sin\theta_{n+1}\,\delta. \tag{5.23}$$

5.7.3 Admissibility

The rover footprint is a disc of radius ρ. A pose is clear if no cell whose centre lies within $\rho + 0.7072\,r$ of the rover centre is occupied, outside the map, or unknown; the extra half cell diagonal makes the test conservative. Unknown cells are tolerated only under the rover's footprint at its starting pose, so that it can drive off its own unobserved underbody, and never when reversing. A candidate is *admissible* if every pose of its rollout, including the braking tail, is clear. Because the tail is included, an admissible command is one the rover can still stop from before reaching anything the map knows about, or anything it has not seen.

5.7.4 Selection

Among admissible candidates the planner minimizes

$$J(v, \omega) = (v - v_i)^2 + 0.3\,(\omega - \omega_i)^2 + \lambda_{\text{rev}} \max(0, -v), \tag{5.24}$$

with ties broken by sample order. If no candidate is admissible the command is zero and the arbiter reports `obstacle_blocked`. If the chosen command differs from the intent by more than 0.15 in either component the decision is labelled `assisted`, which is how the operator station shows that the planner, not the operator or the follower, is shaping the motion. Table 5.3 lists the defaults.

5.8 Control laws for each autonomy level

Each level differs only in how it produces the intent (v_i, ω_i) and in which of the layers above it uses. Figure 5.1 shows the pipeline, and Table 5.4 summarizes it.

Table 5.3: Local planner defaults (`PlannerConfig`).

Symbol	Meaning	Default
ρ	Footprint radius	0.65 m
$v_{\max},\ v_{\text{rev}}$	Forward and reverse speed limits	2.0, 0.15 m/s
$\omega_{\max}$	Yaw rate limit	2.0 rad/s
$a,\ \alpha$	Linear and yaw acceleration	1.0 m/s^2, 2.0 rad/s^2
a_b	Braking deceleration	1.0 m/s^2
H	Prediction horizon	2.0 s (1.0 s in L4 Explore)
λ_{rev}	Reverse penalty	0.05
Δt	Planning step	0.1 s

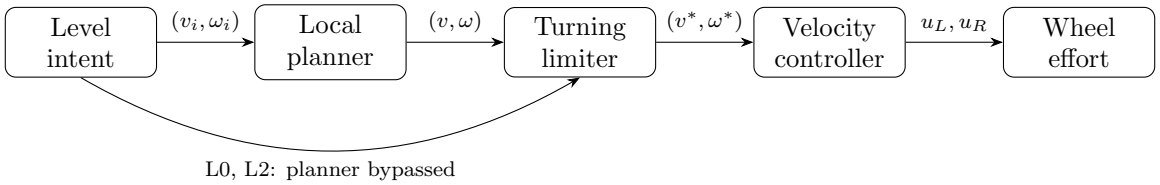


Figure 5.1: Command pipeline. L0 and L2 skip the local planner; every managed command passes the turning limiter; every command passes the velocity controller and its stop conditions.

5.8.1 L0: teleoperation

The intent is the operator's command, limited and checked for freshness:

$$(v_i, \omega_i) = \begin{cases} (\text{clip}(v_{\text{op}}, \pm 2),\ \text{clip}(\omega_{\text{op}}, \pm 2)) & t - t_{\text{op}} < 0.5 \text{ s}, \\ (0, 0) & \text{otherwise.} \end{cases} \tag{5.25}$$

The command bypasses the planner: $(v, \omega) = (v_i, \omega_i)$. From ARGOS it is a managed command, so it still passes the turning limiter and the velocity controller. The operator closes the outer loop, and the 0.5 s lease means a lost link stops the rover within half a second.

5.8.2 L1: assisted teleoperation

The intent is the same operator command, but the planner chooses the motion:

$$(v, \omega) = \arg \min_{(v,\omega) \in \mathcal{A}} J(v, \omega \mid v_{\text{op}}, \omega_{\text{op}}), \tag{5.26}$$

where $\mathcal{A}$ is the admissible set. On an open map the result equals the operator's command up to the acceleration window; near an obstacle the planner slows or steers away, and the decision is labelled `assisted`.

5.8.3 Pursuit toward a goal

L2 and L3 both use a pure-pursuit-style heading law toward a target point $\mathbf{g}$. With $d = \|\mathbf{g} - (x, y)\|$ and bearing error $e_\theta = \mathrm{wrap}(\mathrm{atan2}(g_y - y,\, g_x - x) - \theta)$,

$$\omega_i = \mathrm{clip}(k_\theta e_\theta,\ \pm\omega_{\max}^{\mathrm{wp}}), \tag{5.27}$$

$$v_i = \begin{cases} v_c \left(1 - \dfrac{|e_\theta|}{\gamma}\right) \max\left(\min\left(\dfrac{d}{R_s}, 1\right),\ 0.2\right) & |e_\theta| < \gamma, \\ 0 & |e_\theta| \geq \gamma. \end{cases} \tag{5.28}$$

The rover drives at cruise speed v_c when aligned and far, slows linearly with heading error and stops forward motion beyond the gate γ, and slows to at least 20% of cruise inside the slow radius R_s. Inside the arrival radius R_a it stops, or, if the goal has a final yaw θ_g, turns with $\omega_i = \mathrm{clip}(k_\theta\, \mathrm{wrap}(\theta_g - \theta), \pm\omega_{\max}^{\mathrm{wp}})$ until within 0.12 rad. The defaults are $v_c = 1.0$ m/s, $k_\theta = 1.6$ s^{-1}, $\omega_{\max}^{\mathrm{wp}} = 1.2$ rad/s, $\gamma = 0.55$ rad, $R_s = 3.0$ m, and $R_a = 0.75$ m.

5.8.4 L2: direct waypoint

L2 applies Equation (5.28) straight toward the operator's goal and bypasses the planner, so it does not avoid obstacles. It is the baseline that isolates the value of obstacle avoidance in L3.

5.8.5 L3: waypoint with obstacle avoidance

L3 adds a global route and the local planner. An A* search on the occupancy grid finds a route of free cells from the rover to the goal. The graph is four-connected with unit edge cost, which prevents cutting corners diagonally past obstacles, and a cell is traversable only if the footprint test of Section 5.7 passes there. The heuristic is the Manhattan distance in cells,

$$h(c) = |c_{\mathrm{col}} - g_{\mathrm{col}}| + |c_{\mathrm{row}} - g_{\mathrm{row}}|, \tag{5.29}$$

which is admissible and consistent for unit-cost four-connected motion, so the route is shortest in cell count. The route is rechecked every 0.2 s and whenever the map changes along it. The rover pursues the next route point with Equation (5.28), with the arrival radius tightened to 0.05 m at intermediate corners, and the planner turns that intent into the executed command. Unknown space is never routed through.

5.8.6 L4: frontier exploration

L4 Explore drives to frontiers, free cells next to unknown space, with no per-goal approval, for a time budget T_b set by the operator. A breadth-first search from the rover over the grid visits each reachable cell c with travel distance $n(c)$ in cells. Its information gain $G(c)$ is the number of unknown cells in a square window of half-width $\lceil \rho / r \rceil + 2$ cells around it. The chosen frontier maximizes gain per unit travel,

$$f^* = \arg\max_{c \in \mathcal{F}} \frac{G(c)}{1 + n(c)}, \tag{5.30}$$

over the candidate set $\mathcal{F}$ of cells that pass the footprint test with no unknown-space exception, have $G(c) > 0$, lie more than $\max(\rho, 0.8\text{ m})$ from the rover, and are more than 1 m from any blacklisted point. The frontier becomes a goal followed through the planner, with the horizon reduced to $H = 1$ s so that goals at the edge of the map remain reachable while unknown cells stay blocked. If the rover makes no progress for 5 s, the frontier is blacklisted and another is chosen; after three attempts a point is excluded for the rest of the run. Progress is reported as observed free area and explored fraction,

$$A_{\text{free}} = N_{\text{free}}\, r^2, \qquad \eta = \frac{N_{\text{known}}}{N_{\text{cells}}}. \tag{5.31}$$

The run ends when $t \geq T_b$, when $\mathcal{F}$ is empty, on a stop or takeover, or on a sustained stale map or safety hold. The map-stale ending has a 3 s grace, while motion is still held from the first stale tick (Terra PR #60). The explorer can also ask the arbiter for a look-around, a slow turn on the spot to observe more of the map. That look-around was cleared only inside L4, so after an L4 run the rover kept executing it in L2 and L0 and reported `obstacle_blocked` although those levels never consult the map; changing level now ends it (Terra PR #75). In ARGOS this level is offered as "L4 Explore" with a 3, 5, or 10 minute budget (ARGOS PR #64).

5.8.7 L4: target search

Target search uses frontier navigation inside an operator-defined rectangle, implemented by marking every cell outside the rectangle as occupied, and confirms a target from detector observations $\{(x_k, y_k, c_k, t_k)\}$. A target is confirmed when at least N consecutive fresh observations of the requested class satisfy

$$c_k \geq c_{\min}, \qquad \max_{j,k} \|(x_j, y_j) - (x_k, y_k)\| \leq d_{\text{agree}}, \qquad t_{\text{last}} - t_{\text{first}} \geq \tau_{\text{span}}, \tag{5.32}$$

with no gap or age above τ_{fresh}; a disagreeing observation restarts the set. The defaults are $N = 3$, $c_{\min} = 0.8$, $d_{\text{agree}} = 1$ m, $\tau_{\text{span}} = 0.5$ s, and $\tau_{\text{fresh}} = 0.5$ s. These are software thresholds, not a calibrated detector model.

5.8.8 L5: task autonomy

L5 was defined as the level at which the robot decides what is needed. Its first implementation is narrower: one rover takes a task in plain language from the operator and decides its own waypoints for it (Terra PR #70, ARGOS PR #85, merged October 25, 2026). A language model running on the phone makes the decisions; the Terra core keeps every safety and motion rule. **This level has not been run on a phone.** At the time of writing TerraPhone with the model runner had not been built and run on an iOS 26 device, the ARGOS task panel had not been exercised against a live rover, and no task has been completed end to end. Everything below describes the implemented design, not observed behaviour.

Loop. The operator selects L5 in ARGOS and types a task of at most 500 characters, with a time budget (default 300 s, at most 3600 s) and a limit on model calls (default 20, at most

100). The arbiter then repeats one step until the task ends:

1. It builds a prompt from the occupancy map: the whole local map reduced to a symbolic grid of 1 m cells (a cell is occupied if any of its 0.1 m cells is occupied, free only if all of them are observed and free, and unknown otherwise), up to six reachable frontiers, up to eight obstacle blobs and eight landmarks, each with an identifier, the rover's pose, and the six most recent steps of history.

2. The phone runs the model on that prompt. The rover holds while the model thinks; the model runs off the control queue, so the 10 ms tick and the 20 Hz motor output do not wait for it.

3. The arbiter parses the reply, which must be one JSON object choosing one of four decisions: `go` (up to five waypoints, given as identifiers or local points), `ask_operator` (a question shown in ARGOS; the rover waits for the answer), `done`, or `fail`.

4. Each waypoint is snapped to observed, footprint-clear free space within 0.5 m, and must have an A* route (Section 5.7) before the waypoint follower accepts it. Accepted waypoints are driven exactly as in L3.

A reply that contains a speed, an effort or a servo command is rejected: the model chooses *where*, never *how*. A waypoint whose route stays blocked for 4 s, or that is not reached within 90 s, is abandoned and the model is asked again. A query with no answer after 90 s counts as a failed step, and three consecutive failed steps (invalid replies, timeouts or model errors) fail the task. The operator can answer a question or add guidance at any time, which replaces the current plan, and can cancel; a stop, a level change or a goal cancel also ends the task.

Model. The model runner uses Apple's on-device Foundation Models framework (iOS 26 or later), with text input only. The level is offered to ARGOS only while the phone reports that a model is available. The runner sits behind a small protocol so that another on-device model, for example a vision–language model run through MLX or llama.cpp, can replace it. [**TODO: Run L5 end to end on an iOS 26 iPhone, first in Zorvane and then on CrabTrapBot, and report model latency per step, the fraction of replies rejected and why, and task completion for a small set of fixed tasks.**]

Relation to the rest of the ladder. L5 adds a decision maker above L3, not a new controller: the A* check, the local planner of Section 5.7, the turning limiter and every safety hold apply unchanged. Because the model only proposes waypoints, a bad reply costs time, not safety. The pilot study does not use L5. Deciding what the *fleet* needs, the original meaning of the level, is not implemented; Chapter 6 discusses the recommendation system that would sit between the operator and such decisions.

Table 5.4: How each level produces motion. "Managed" commands pass the turning limiter.

Level	Intent (v_i, ω_i)	Local planner	Human role
L0 Teleop	Operator command, 0.5 s lease	Bypassed	Closes the loop
L1 Assisted	Operator command	Eq. (5.24)	Drives; planner vetoes
L2 Direct	Pursuit, Eq. (5.28), to goal	Bypassed	Sets goal; watches
L3 Waypoint	Pursuit along A* route	Eq. (5.24)	Sets goal
L4 Explore	Pursuit to f^*, Eq. (5.30)	Eq. (5.24), $H = 1$ s	Sets budget
L4 Search	Pursuit to frontier in bounds; confirmation rule	Eq. (5.24)	Sets area, class, budget
L5 Task	Pursuit along A* route to model-chosen waypoints	Eq. (5.24)	Gives task; answers questions

5.9 Outdoor guidance and heading fusion

5.9.1 Geographic goals

Outdoors, the goal is a latitude and longitude. From the current fix (φ_1, λ_1) to the goal (φ_2, λ_2), Terra computes the great-circle distance and initial bearing with the haversine formula, using $R_\oplus = 6\,371\,000$ m:

$$h = \sin^2\frac{\Delta\varphi}{2} + \cos\varphi_1 \cos\varphi_2 \sin^2\frac{\Delta\lambda}{2}, \qquad d = 2R_\oplus \arcsin\sqrt{h}, \tag{5.33}$$

$$\beta = \operatorname{atan2}(\sin\Delta\lambda\cos\varphi_2,\ \cos\varphi_1\sin\varphi_2 - \sin\varphi_1\cos\varphi_2\cos\Delta\lambda)\,. \tag{5.34}$$

The bearing β (degrees clockwise from north) is converted into a local yaw ψ using the fused heading of the next subsection, and a local target is placed at most 3 m ahead:

$$\mathbf{g} = (x, y) + \min(d, 3)\,(\cos\psi, \sin\psi). \tag{5.35}$$

That target is refreshed at most twice per second and followed exactly as in L3, so the local planner still checks every command; forward speed is capped at 0.3 m/s. Arrival requires three distinct fixes within the arrival radius. For goals given in a local frame with a geographic origin (φ_0, λ_0), Terra uses an equirectangular projection, $x_{\text{east}} = (\lambda - \lambda_0)\,k\cos\varphi_0$ and $y_{\text{north}} = (\varphi - \varphi_0)\,k$, with $k = 40\,075\,016.686/360$ m per degree.

5.9.2 Heading fusion

GPS gives position, not heading, and the phone's compass near the motors was off by 30–50°. Terra therefore estimates a single slowly varying offset c (degrees) between the local VIO frame and true north, such that the true heading is

$$h = c - \theta^\circ, \tag{5.36}$$

where θ° is the local yaw in degrees (local yaw is counter-clockwise; true heading is clockwise). VIO carries the heading between corrections, and the filter only has to track c. It is a scalar Kalman filter with random-walk process noise. Between corrections the variance grows as

$$P(t) = P_k + q\,(t - t_k), \qquad q = 0.25 \text{ deg}^2/\text{s}. \tag{5.37}$$

Two measurements of c are available. A compass sample h_c with reported accuracy σ_c gives $z = h_c + \theta^\circ$ with $R = \sigma_c^2$. A GPS course-over-ground sample χ gives $z = \chi + \text{atan2}(\Delta y, \Delta x)^\circ$, pairing it with the local direction of travel over the same 1 s window, with $R = \max(\sigma_\chi, 2^\circ)^2$; it is used only at 0.5 m/s or faster, with less than 10° of turning in the window. For a measurement z, the innovation and update are

$$\nu = \text{wrap}_{180}(z - c), \tag{5.38}$$

$$K = \frac{P}{P + R}, \qquad c \leftarrow c + K\nu, \tag{5.39}$$

$$P \leftarrow \begin{cases} \dfrac{PR}{P+R} & \text{GPS course,} \\ \min\left(P,\ \max\left(\dfrac{PR}{P+R},\ R\right)\right) & \text{compass.} \end{cases} \tag{5.40}$$

The compass update floors the variance at R, because compass error is mostly a bias that does not average out; a compass sample is ignored entirely when $R > 4P$. A measurement is rejected if $\nu^2 > 9(P + R)$, a three-sigma gate; after ten consecutive rejections the filter restarts from the latest measurement. The filter also restarts if the VIO pose has a gap longer than 2 s or a yaw step faster than 3 rad/s. The reported heading accuracy is $\sqrt{P(t)}$; ARGOS holds an outdoor mission when it exceeds 35°, and Terra requires it below 30° to admit one.

5.10 Multi-rover map fusion (prototype)

Every rover in Section 5.4 maps in its own frame, and nothing on the rovers combines those maps. A first step towards a shared map is a two-rover prototype, `scripts/plot_sim_rovers.py` in the Terra repository, which registers one rover's occupancy grid onto another's and shows the fused result. [**TODO: The script is not yet committed to Terra (checked Oct 26, 2026); this section was checked against the working copy Yojan shared that day. Commit it and pin the revision in the header comment.**] It is a visualisation and registration tool only. It runs as a separate Python process that subscribes over Zenoh, it does not publish the fused grid back, and it does not change either rover's mapper or the arbiter.

5.10.1 Inputs and accumulation

The prototype subscribes to the `pose` and `map/occupancy` topics of a reference rover R and a source rover S (by default IDs 0 and 1), and estimates the rigid transform that carries S's map into R's frame. The occupancy packet is the one Terra already publishes (Section 5.4): values -1 for unknown and 0–100 otherwise, rows along world $+y$, origin at the minimum cell corner, and the same threshold $P \geq 65$ for occupied. The pose is drawn on the plot but is not

a registration measurement; the transform comes from map structure alone.

Each published grid is a rolling window, so cells fall out of it as the rover moves. The prototype keeps a persistent map per rover, keyed by integer lattice coordinates. A cell at column j and row k of a grid with origin (o_x, o_y) and resolution r has key

$$(i_x, i_y) = (\mathrm{round}(o_x/r) + j,\ \mathrm{round}(o_y/r) + k)\,. \tag{5.41}$$

This is exact for Terra's mapper, which only ever shifts its origin by whole cells. The newest known value at a key replaces the old one, unknown cells never erase evidence, and older or repeated map revisions are ignored. A change of run ID clears both maps and the current alignment. When a persistent map would exceed 5×10^5 cells it is coarsened by an integer factor $s = \max(2, \lceil \sqrt{N_x N_y / 5 \times 10^5} \rceil)$, keeping the maximum value in each bin, which favours obstacles.

5.10.2 Registration

Registration uses only occupied cells. Their centres form point sets P_R and P_S. Each set is downsampled to one point per voxel of width $\max(0.25\ \mathrm{m}, 2r)$ and capped at 250 points, so that long walls do not dominate.

Global search. Before an alignment exists, and then every 30 s, the prototype sweeps yaw θ in 10° steps over the full circle. At each θ, every pair of a reference point and a rotated source point votes for a translation,

$$\mathbf{d}_{ij} = \mathbf{p}_{R,i} - R(\theta)\,\mathbf{p}_{S,j}, \tag{5.42}$$

quantised into bins of width $b = \max(0.25\ \mathrm{m}, 3r)$. The two fullest bins at each yaw become candidates. A candidate is scored by nearest-neighbour matching within $\rho = \max(0.35\ \mathrm{m}, 4r)$, with each reference point matched at most once. With M unique matches, B occupied 1 m bins containing match midpoints, and RMS match distance e, candidates are ranked by larger M, then larger B, then smaller e, then more votes. This is an exhaustive discrete search, not RANSAC.

Refinement. The best candidate, or between global searches the current alignment, seeds point-to-point ICP on up to 6 000 occupied points per map. Each iteration pairs every transformed source point with its nearest reference point, drops pairs farther than $\max(0.65\ \mathrm{m}, 6r)$, trims the rest to the 75th percentile of distance, and requires at least eight pairs. The rotation is the least-squares solution from the SVD of

$$H = \sum_i (\mathbf{x}_i - \bar{\mathbf{x}})(\mathbf{y}_i - \bar{\mathbf{y}})^{\mathsf{T}}, \tag{5.43}$$

and the translation aligns the centroids. ICP stops after 35 iterations or when the update falls below 10^{-4} m and 10^{-5} rad. The refined transform is then rescored on the downsampled sets at radius ρ.

5.10.3 Scoring and acceptance

With precision $P_m = M/N_S$ and recall $Q_m = M/N_R$ over the downsampled point counts, the prototype computes a confidence

$$C = \mathrm{clip}_{[0,1]}\Big(0.55\,F_1 + 0.20\min(1, B/8) + 0.20\,e^{-e/\rho} + 0.05\,A_K\Big), \qquad F_1 = \frac{2P_mQ_m}{P_m + Q_m}, \quad (5.44)$$

where A_K is the mean agreement $1 - |c_R - c_S|/100$ over cells known in both maps, computed only during a global search. It also computes an evidence score

$$I = \mathrm{clip}_{[0,1]}\Big(0.40\min(1, \tfrac{M}{40}) + 0.25\min(1, \tfrac{B}{8}) + 0.20\min(1, \tfrac{\min(N_R,N_S)}{100}) + 0.15\min(1, \tfrac{N_{\mathrm{ICP}}}{50})\Big), \quad (5.45)$$

and their product $J = CI$. The weights are hand-chosen. C is a ranking score, not a calibrated probability that the alignment is correct, and the thesis does not use it as one.

A candidate is eligible if $M \geq 6$, $B \geq 2$, $e \leq 0.45$ m, $C \geq 0.18$ and $I \geq 0.025$. The first eligible candidate becomes the incumbent. A later one replaces it only if C rises by more than 10^{-4}, I does not fall, and J rises. A rejected candidate leaves the current fused map in place. Alignment is attempted at most every 2 s.

5.10.4 Fusion

The fused grid is recomputed from the two persistent maps under the incumbent transform, never from the previous fused grid, so no observation is counted twice. It covers the union of both extents at the finer input resolution, coarsened if needed to stay within 5×10^5 cells. Each known value c becomes $p = \mathrm{clip}(c/100, 0.01, 0.99)$, and the two maps are combined in log odds,

$$\ell_F = \mathrm{clip}\left(\sum_{m\in\{R,S\}} \ln\frac{p_m}{1-p_m},\ \pm 4\right), \qquad P_F = \mathrm{round}\left(\frac{100}{1 + e^{-\ell_F}}\right), \quad (5.46)$$

mirroring the single-rover update and its clamp. A cell unknown in both maps stays -1. The sum treats the two rovers' observations of a cell as independent evidence. Fusion runs at most every 0.75 s and the plot at most at 2 Hz.

5.10.5 Validation so far

In Zorvane all rovers report maps in one shared world frame (Chapter 4), so the correct transform between two simulated rovers is the identity. To test registration on simulated data the prototype therefore applies a known transform $A = (2.0\text{ m}, -1.5\text{ m}, 23°)$ to the source map by default and reports the residual between the estimate and A^{-1}; `--no-test-transform` disables it for maps that are genuinely in different frames, as two phones' VIO frames are. A built-in self-test registers a synthetic asymmetric 24×20 m layout at 0.1 m under three known transforms and passes if each alignment is eligible, each residual is within 0.30 m and 4°, and the fused map keeps at least 70% of the reference map's known cells. Table 5.5 gives the result of one run.

Applied transform			Residual		
x (m)	y (m)	yaw	translation (m)	yaw	M
2.0	−1.5	23°	0.028	0.97°	160
−3.0	2.0	−31°	0.035	0.60°	159
0.8	1.2	0°	0.199	0.01°	170

Table 5.5: Prototype self-test on synthetic data (run Oct 26, 2026). M is the number of unique occupied-cell matches after ICP.

The rotated cases recover the transform to a few centimetres. The pure translation, which should be the easiest, is the worst: ICP converges two cells short along x with an RMS match distance of only 0.01 m, because points on long axis-aligned walls match exactly whatever their position along the wall, and the 75th-percentile trim discards the wall ends that would correct it. It still passes, but with only 0.10 m to spare. The coverage check passes for any transform, since the reference map alone supplies its known cells, so it guards against coarsening losses and not against a wrong merge. This is a software check on synthetic data, not evidence of performance on real maps. [**TODO: Report registration residuals from the Zorvane two-rover run with the test transform, and from two phones in one room with `--no-test-transform` against OptiTrack or tape-measured ground truth.**]

5.10.6 Limitations

- **Uncalibrated scores.** C, I and J rank candidates; none is a probability of a correct merge.
- **Frame stability.** Accumulation assumes each rover's frame does not jump. An ARKit relocalisation or VIO reset without a new run ID would corrupt that rover's persistent map.
- **Ratcheting incumbent.** A new alignment, local or global, replaces the incumbent only if its confidence is strictly higher. A wrong early merge with a high score is therefore kept until the run changes, and since the incumbent was scored on smaller maps, a correct transform rescored on grown maps can be rejected because recall over the larger point sets fell.
- **Static world.** Old observations persist until the cell is seen again, and max-coarsening keeps stale obstacles.
- **Ambiguity.** Translation voting can lock onto a wrong but plausible alignment in symmetric or repetitive spaces such as corridors; the spatial-spread term reduces this but does not remove it.
- **Two rovers.** It registers one source to one reference. It is not a multi-robot pose graph and has no loop closure.
- **Not on the robots.** The fused map reaches no rover, planner, or ARGOS. Exploration still runs on each rover's own map, so two rovers can explore the same area.

Publishing the fused map back, so that the frontier explorer of Section 5.8 can avoid space another rover has already covered, is future work.

5.11 Summary

Each autonomy level is a different source of intent feeding one shared pipeline: a dynamic-window planner that only accepts commands it can still stop from, a turning limiter that keeps skid-steer turns gentle, and a feedforward PI velocity controller on VIO and IMU estimates, with every stage stopping on stale input. L0 and L2 bypass the planner, which makes them the baselines for measuring what obstacle avoidance adds; L1, L3, L4, and L5 share it. L5 puts an on-phone language model above L3 to choose waypoints for a plain-language task, with every waypoint checked by A* and driven on the L3 path; it is implemented but untested on hardware. The occupancy map's ground plane is level and placed at the mount-calibrated camera height, a correction found by inspecting labelled point clouds streamed to ARGOS, and ARKit's classified floor mesh now vetoes obstacle returns that lie on the floor; wall, window and door mesh now stops rays at glass, though this is untested on real glass. Against motion capture, the phone's VIO speed was steady, low-noise and correctly oriented but read about 9% low in the capture room, which the speed loop tolerates and absolute distances do not. The faults that most affected driving in October 2026 were timing faults on the phone's control queue, not errors in the control laws: each stale input correctly produced a hold, and the holds compounded. Outdoors, a scalar Kalman filter on the VIO-to-north offset fuses compass and GPS course into the heading that turns a geographic bearing into a local goal. A separate two-rover prototype registers one rover's occupancy grid onto another's by occupied-cell voting and ICP and fuses them in log odds, but only for display; no rover uses the fused map yet. **[TODO: Add a figure of a recorded planner trace (candidates and selected arc) from a real run.]** **[TODO: Report step responses of the velocity controller on the real rover once it is tuned.]**

Bibliography

[1] NEXT Competition 2026 Rules. Rules document dated August 7, 2026, as encoded in the Zorvane simulator (simulator/NEXT.md), 2026. VERIFY organizer, author, and where the rules are published; cite the document directly.

[2] Evan Ackerman. DARPA SubT finals: Robot operator wisdom. IEEE Spectrum, 23 September 2021. `https://spectrum.ieee.org/darpa-subterranean-challenge-operator`, 2021.

[3] Avian contributors. Avian: ECS-driven physics for Bevy. `https://github.com/Jondolf/avian`, 2026. VERIFY URL and version. Accessed October 2026.

[4] Jenay M. Beer, Arthur D. Fisk, and Wendy A. Rogers. Toward a framework for levels of robot autonomy in human-robot interaction. *Journal of Human-Robot Interaction*, 3(2): 74–99, 2014. VERIFY.

[5] Bevy contributors. Bevy: A data-driven game engine built in Rust. `https://bevyengine.org`, 2026. VERIFY version used by Zorvane. Accessed October 2026.

[6] Jessie Y. C. Chen and Michael J. Barnes. Human-agent teaming for multirobot control: A review of human factors issues. *IEEE Transactions on Human-Machine Systems*, 44(1): 13–29, 2014. VERIFY.

[7] Jacob W. Crandall and M. L. Cummings. Identifying predictive metrics for supervisory control of multiple robots. *IEEE Transactions on Robotics*, 23(5):942–951, 2007. VERIFY.

[8] Jacob W. Crandall, Michael A. Goodrich, Dan R. Olsen, and Curtis W. Nielsen. Validating human-robot interaction schemes in multitasking environments. *IEEE Transactions on Systems, Man, and Cybernetics – Part A: Systems and Humans*, 35(4):438–449, 2005. VERIFY.

[9] M. L. Cummings and Paul J. Mitchell. Predicting controller capacity in supervisory control of multiple UAVs. *IEEE Transactions on Systems, Man, and Cybernetics – Part A: Systems and Humans*, 38(2):451–460, 2008. VERIFY.

[10] Defense Advanced Research Projects Agency. DARPA subterranean challenge winners. `https://www.darpa.mil/news/2021/subterranean-challenge-winners`, 2021. News release, September 2021.

[11] Jeffrey Delmerico and Davide Scaramuzza. A benchmark comparison of monocular visual-inertial odometry algorithms for flying robots. In *IEEE International Conference on Robotics and Automation (ICRA)*, pages 2502–2509, 2018. VERIFY pages.

[12] M. Bernardine Dias, Robert Zlot, Nidhi Kalra, and Anthony Stentz. Market-based multirobot coordination: A survey and analysis. *Proceedings of the IEEE*, 94(7):1257–1270, 2006.

[13] Mica R. Endsley. Toward a theory of situation awareness in dynamic systems. *Human Factors*, 37(1):32–64, 1995. VERIFY.

[14] Dieter Fox, Wolfram Burgard, and Sebastian Thrun. The dynamic window approach to collision avoidance. *IEEE Robotics & Automation Magazine*, 4(1):23–33, 1997.

[15] Michael A. Goodrich, Dan R. Olsen, Jacob W. Crandall, and Thomas J. Palmer. Experiments in adjustable autonomy. In *IJCAI Workshop on Autonomy, Delegation and Control: Interacting with Intelligent Agents*, 2001. VERIFY.

[16] Google. ARCore fundamental concepts. `https://developers.google.com/ar/develop/fundamentals`, 2026. Accessed October 2026.

[17] Sandra G. Hart and Lowell E. Staveland. Development of NASA-TLX (task load index): Results of empirical and theoretical research. In Peter A. Hancock and Najmedin Meshkati, editors, *Human Mental Workload*, pages 139–183. North-Holland, 1988. VERIFY.

[18] International Tennis Federation. Rules of tennis, appendix i: The ball, 2026. VERIFY edition year and URL.

[19] Pyojin Kim, Jungha Kim, Minkyeong Song, Yeoeun Lee, Moonkyeong Jung, and Hyeong-Geun Kim. A benchmark comparison of four off-the-shelf proprietary visual–inertial odometry systems. *Sensors*, 22(24):9873, 2022. doi: 10.3390/s22249873.

[20] Nathan Koenig and Andrew Howard. Design and use paradigms for Gazebo, an open-source multi-robot simulator. In *Proceedings of the IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pages 2149–2154, 2004. VERIFY.

[21] Paul Milgram and Fumio Kishino. A taxonomy of mixed reality visual displays. *IEICE Transactions on Information and Systems*, E77-D(12):1321–1329, 1994.

[22] Mozilla. UniFFI: A multi-language bindings generator for Rust. `https://mozilla.github.io/uniffi-rs/`, 2026. Accessed October 2026.

[23] Robin R. Murphy. Rescue robotics for homeland security. *Communications of the ACM*, 47(3):66–68, 2004. URL `https://cacm.acm.org/research/rescue-robotics-for-homeland-security/`. VERIFY volume, number and pages.

[24] Robin R. Murphy. *Disaster Robotics*. MIT Press, 2014. VERIFY.

[25] Robin R. Murphy et al. Robots and drones at the Surfside building collapse. IEEE Spectrum, August 2021. `https://spectrum.ieee.org/building-collapse-surfside-robots`, 2021. VERIFY exact title and author list.

[26] Keiji Nagatani et al. Redesign of rescue mobile robot Quince: Toward emergency response to the nuclear accident at Fukushima Daiichi nuclear power station on March 2011. In *IEEE International Symposium on Safety, Security, and Rescue Robotics (SSRR)*, 2011. URL `https://k-nagatani.org/pdf/2011-SSRR-KEI-Rev-online.pdf`. VERIFY title, full author list and pages from the PDF.

[27] Dan R. Olsen and Michael A. Goodrich. Metrics for evaluating human-robot interactions. In *Performance Metrics for Intelligent Systems Workshop (PerMIS)*, 2003. VERIFY.

[28] Dan R. Olsen and Stephen Bart Wood. Fan-out: Measuring human control of multiple robots. In *ACM Conference on Human Factors in Computing Systems (CHI)*, pages 231–238, 2004. VERIFY.

[29] OpenStreetMap contributors. Field #3, west campus, george mason university (way 233194680). `https://www.openstreetmap.org/way/233194680`, 2026. Data available under the ODbL. Accessed October 2026.

[30] Raja Parasuraman, Thomas B. Sheridan, and Christopher D. Wickens. A model for types and levels of human interaction with automation. *IEEE Transactions on Systems, Man, and Cybernetics – Part A: Systems and Humans*, 30(3):286–297, 2000. VERIFY.

[31] Lynne E. Parker. ALLIANCE: An architecture for fault tolerant multirobot cooperation. *IEEE Transactions on Robotics and Automation*, 14(2):220–240, 1998.

[32] SAE International. Taxonomy and definitions for terms related to driving automation systems for on-road motor vehicles. Technical Report J3016, SAE International, 2021. VERIFY revision year.

[33] Shital Shah, Debadeepta Dey, Chris Lovett, and Ashish Kapoor. AirSim: High-fidelity visual and physical simulation for autonomous vehicles. In *Field and Service Robotics*, 2017. VERIFY venue year and pages.

[34] Thomas B. Sheridan. *Telerobotics, Automation, and Human Supervisory Control*. MIT Press, 1992. VERIFY.

[35] Thomas B. Sheridan and William L. Verplank. Human and computer control of undersea teleoperators. Technical report, MIT Man-Machine Systems Laboratory, 1978. VERIFY.